
Algorithm Engineering

La parte di Teoria, risolta

Teoremi, dimostrazioni e analisi di complessità
con la risoluzione dettagliata di *tutte* le domande teoriche
dei temi d'esame 2020–2025

Università di Pisa — Laurea Magistrale in Informatica.

Compendio costruito a partire dalla dispensa del corso, dal testo Pearls of Algorithm Engineering (Ferragina, 2023) e da 40 temi d'esame. Per ogni argomento: richiamo di teoria autosufficiente, elenco delle domande effettivamente uscite (con date), e soluzione completa con tutte le dimostrazioni e i conti di complessità.

Obiettivo: padroneggiare la parte di teoria. Studia i *Richiami*, poi verifica di saper rispondere a freddo a ogni *Domanda d'esame* coprendo l'eventuale *Soluzione*.

Indice

Come usare questo documento	3
1 Modelli di calcolo e complessità I/O	4
1.1 Bound fondamentali di sorting e permuting	4
2 Warm-up del corso: Maximum Subarray e List Ranking	7
2.1 Maximum Subarray Sum	7
2.2 List Ranking	8
3 Ordinamento: tecniche, lower bound e varianti	9
3.1 Snow Plow: lunghezza attesa del run $= 2M$	9
3.2 Bounded QuickSort (profondità di stack $O(\log n)$)	10
3.3 Lower bound di ordinamento (modello a due livelli)	11
3.4 Ordinamento di stringhe: lower bound, RadixSort, Multikey QuickSort	11
4 Random Sampling e Reservoir Sampling	14
4.1 Reservoir Sampling (n ignoto)	14
5 Intersezione di insiemi ordinati	16
5.1 Mutual Partitioning	16
5.2 Doubling / Exponential (Galloping) Search	16
5.3 2-Level Intersection (modello a due livelli)	17
5.4 Intersezione di liste compresse con Elias-Fano	18
6 Dizionari: hashing, perfect hashing, cuckoo, Bloom, Skip List	19
6.1 Hashing universale	19
6.2 Perfect Hashing (statico)	20
6.3 Cuckoo Hashing	20
6.4 Bloom Filter	21
6.5 Skip List	22
7 Strutture dati randomizzate: Treap (e Skip List)	24
7.1 Treap: definizione	24
7.2 Operazioni (con rotazioni)	24
8 Prefix Search: trie, front coding, interpolation search	26
8.1 Memorizzazione delle stringhe e Front Coding	26
8.2 Trie compatta vs Patricia trie	26
8.3 Interpolation Search	27
9 Substring Search: Suffix Array, LCP, Suffix Tree	29
9.1 Definizioni: Suffix Array e LCP array	29
9.2 Ricerca del pattern e COUNT	29
9.3 Costruzione del Suffix Array	30
9.4 Suffix Tree	32

10 Integer Coding: numero di bit emessi	33
10.1 Formule di lunghezza e codifiche bit-per-bit	33
10.2 Codice interpolativo (esempio svolto)	34
10.3 Elias–Fano: spazio, esempio svolto e Access	35
10.4 Tabella di confronto: bit emessi	35
11 Statistical Coding: entropia, Huffman, Arithmetic	37
11.1 Huffman e sua ottimalità	37
11.2 Canonical Huffman	38
11.3 Arithmetic Coding	39
12 Compressione basata su dizionario: LZ77, LZ78, LZW	42
12.1 LZ77 (sliding window) e gzip	42
12.2 LZ78 (dizionario esplicito a trie)	43
12.3 LZW (solo id, dizionario inizializzato)	43
12.4 LZ77 vs LZ78/LZW (confronto)	44
13 Burrows–Wheeler Transform e pipeline bzip2	45
13.1 Trasformata diretta e inversa	45
14 Strutture dati compresse: Rank/Select, FM-index, alberi succinti	47
14.1 Rank in $O(1)$ con $o(m)$ bit extra	47
14.2 FM-index: Backward Search	48
14.3 Alberi succinti (LOUDS / parentesi bilanciate)	49

Come usare questo documento

Ogni capitolo è una *scheda* su un argomento e ha tre componenti grafiche:

- **Richiami di teoria** (riquadro blu): definizioni, teoremi, dimostrazioni e bound di complessità, scritti in modo da bastare da soli.
- **Domanda d'esame** (riquadro arancione): la domanda teorica così come è uscita, con le date degli appelli in cui è comparsa.
- **Soluzione svolta** (riquadro verde): la risposta completa, con la dimostrazione e l'analisi che il docente si aspetta.

La sigla **[AAMMGG]** indica l'appello (es. **[240115]** = 15 gennaio 2024). Le domande sono raggruppate per argomento e deduplicate: la stessa domanda uscita in più appelli è risolta una volta sola, segnalando le varianti numeriche (es. Snow Plow con probabilità $1/4$ invece di $1/2$).

Mappa delle domande più frequenti (ordina lo studio per priorità):

#	Domanda teorica	Appelli
1	Struttura succinta per Rank in $O(1)$: descrizione + spazio in bit	7
2	Reservoir Sampling: pseudocodice + correttezza m/n	6
3	Bloom filter: probabilità di bit a 0 + errore di falso positivo	6
4	2-Level set intersection: descrizione + complessità	6
5	Complessità I/O di sorting/permuting nel modello a due livelli	6
6	Hashing universale: definizione + esempio + dimostrazione	5
7	LSD RadixSort: complessità + correttezza	4
8	Snow Plow: lunghezza attesa del run = $2M$ (+ varianti)	4
9	Canonical Huffman: motivazione + preambolo + decodifica	4
10	Suffix Array + LCP: definizioni e ricerca	4

1 Modelli di calcolo e complessità I/O

Richiami di teoria — i tre modelli

Modello RAM. Memoria *illimitata* ad *accesso uniforme*: ogni operazione aritmetica/logica e ogni *accesso* a una cella costano $O(1)$. È il modello classico, ma *ignora la gerarchia di memoria*: in pratica un accesso a un dato in cache costa ordini di grandezza meno di un accesso a disco.

Modello a due livelli (External Memory, EM). Si introducono tre parametri:

- M = numero di elementi che entrano in *memoria interna* (veloce);
- B = numero di elementi per *pagina/blocco* (l'unità di trasferimento);
- N (o n) = numero di *elementi del problema*.

La *complessità I/O* di un algoritmo è il *numero di trasferimenti di blocchi fra disco e memoria interna*; il lavoro in CPU si conta a parte (modello RAM). Un singolo I/O sposta esattamente B elementi. Con D dischi in parallelo, *un I/O sposta $D \cdot B$ elementi*. Due principi guida: *località spaziale* (accedere a dati vicini, già portati in memoria dalla stessa pagina) e *località temporale* (sfruttare al massimo un dato finché è in memoria interna).

Modello Cache-Oblivious. L'algoritmo *non conosce M e B* , eppure risulta *I/O-ottimo su qualunque configurazione di memoria*. Vantaggio: portabilità fra i vari livelli della gerarchia (L1/L2/L3/RAM/disco) senza ri-tuning.

1.1 Bound fondamentali di sorting e permuting

Richiami di teoria — i bound di ordinamento nel modello a due livelli

Definiamo la quantità centrale del corso:

$$\text{sort}(n) = \Theta\left(\frac{n}{B} \log_{M/B} \frac{n}{M}\right) \text{ I/O.}$$

Teorema 1.1 (Limite superiore — Multiway MergeSort). *Si possono ordinare n elementi atomici con $\text{sort}(n) = \Theta\left(\frac{n}{B} \log_{M/B} \frac{n}{M}\right) \text{ I/O}$ e $O(n \log n)$ confronti.*

Teorema 1.2 (Limite inferiore di ordinamento). *Ogni algoritmo comparison-based richiede $\Omega\left(\frac{n}{DB} \log_{M/B} \frac{n}{DB}\right) \text{ I/O}$.*

Teorema 1.3 (Permuting). *Permutare n elementi costa, in alto, $O\left(\min\left(n, \frac{n}{B} \log_{M/B} \frac{n}{M}\right)\right) \text{ I/O}$, e in basso $\Omega\left(\min\left(\frac{n}{D}, \frac{n}{DB} \log_{M/B} \frac{n}{DB}\right)\right) \text{ I/O}$.*

Conseguenza chiave. Quando $B = \Omega\left(\log_{M/B}(n/M)\right)$ (caso reale, B grande), i due bound coincidono: *ordinare e permutare hanno la stessa complessità I/O*. È qui la differenza con il modello **RAM**, dove *permutare costa $\Theta(n)$ e ordinare $\Theta(n \log n)$* : in EM il collo di bottiglia non sono i confronti ma i trasferimenti di pagine, e “mettere ogni elemento al suo posto” è già costoso quanto ordinare.

Perché il numero di passate è $\log_{M/B}(n/M)$

Richiami di teoria — analisi del Multiway MergeSort

Generazione dei run. Si caricano in memoria M elementi alla volta, si ordinano internamente e si scrivono su disco: si ottengono n/M run ordinati, ognuno di lunghezza M , con costo $O(n/B)$ I/O (lettura + scrittura sequenziale di tutto).

Fusione k -way. In memoria interna ci stanno M/B blocchi: se ne usa uno per ciascuno dei k run in input più uno per l'output, quindi

$$k = \frac{M}{B} - 1 = \Theta\left(\frac{M}{B}\right).$$

Per trovare il minimo fra i k candidati si usa un min-heap di k coppie (elemento, indice del run): $O(\log k)$ per elemento in CPU, e $O(z/B)$ I/O per fondere k run di taglia totale z (lettura+scrittura sequenziali).

Numero di livelli. La ricorsione di fusione è un albero k -ario con n/M foglie (i run iniziali). L'altezza, cioè il numero di passate, è

$$\#passate = \log_k \frac{n}{M} = \log_{M/B} \frac{n}{M}.$$

Ogni passata legge e riscrive tutti gli n elementi a $O(n/B)$ I/O. Moltiplicando: $\text{sort}(n) = O\left(\frac{n}{B} \log_{M/B} \frac{n}{M}\right)$. In pratica, con $B = 4\text{KB}$ e $M = 4\text{GB}$ si ha $M/B \approx 2^{20}$: bastano 1–2 passate per dataset di decine di GB.

Disk striping con D dischi

Richiami di teoria — Disk Striping

Con D dischi che lavorano in parallelo, la tecnica di *disk striping* tratta i D dischi come un unico disco logico con pagina di taglia $D \cdot B$: il blocco logico i -esimo è diviso in D “strisce” da B elementi, una per disco, lette/scritte simultaneamente in un solo I/O. Sostituendo $B \rightarrow DB$ nel bound di Multiway MergeSort:

$$\Theta\left(\frac{n}{DB} \log_{M/(DB)} \frac{n}{M}\right) \text{ I/O.}$$

Confronto con l'ottimo. Questo bound *non* è quello ottimo del Teorema 1.2: la base del logaritmo è $M/(DB)$ invece di M/B . La differenza è trascurabile finché D è piccolo rispetto a M/B (caso pratico), ma per D grande il disk striping perde un fattore rispetto allo schema ottimo che distribuisce i run sui dischi in modo indipendente. Lo si accetta perché è semplice e robusto.

Domanda d'esame — I/O di sorting e permuting

State the I/O-complexity of sorting n items in a model with internal memory M , page size B and D disks; compare with the optimal bound and discuss the difference. Then describe disk striping and give the I/O-complexity of multiway mergesort over D disks.

Soluzione svolta

Sorting, un disco. Il Multiway MergeSort ordina con $\text{sort}(n) = \Theta\left(\frac{n}{B} \log_{M/B} \frac{n}{M}\right)$ I/O (Teorema 1.1), e questo è *ottimo* perché coincide con il limite inferiore $\Omega\left(\frac{n}{B} \log_{M/B} \frac{n}{B}\right)$ del Teorema 1.2 (per $D = 1$): infatti $\log_{M/B} M = \Theta(\log_{M/B} B)$, quindi n/M e n/B dentro il log sono equivalenti.

Sorting, D dischi — ottimo. Il limite inferiore diventa $\Omega\left(\frac{n}{DB} \log_{M/B} \frac{n}{DB}\right)$: il parallelismo dei D dischi può ridurre il numero di I/O al più di un fattore D (ogni I/O sposta $D \cdot B$ elementi).

Sorting, D dischi — disk striping. Trattando i D dischi come pagina logica DB , il Multiway MergeSort fa $\Theta\left(\frac{n}{DB} \log_{M/(DB)} \frac{n}{M}\right)$ I/O.

Differenza con l'ottimo. Il disk striping ha base del log $M/(DB)$, l'ottimo ha M/B : lo striping paga un fattore $\frac{\log(M/B)}{\log(M/(DB))}$. È praticamente 1 quando $DB \ll M$ (cioè pochi dischi e M grande), per cui in pratica si usa lo striping per la sua semplicità; per D molto grande servirebbe lo schema ottimo che assegna interi run a dischi distinti per evitare di sincronizzare tutte le strisce a ogni passata.

Permuting. Con D dischi, $\Omega\left(\min\left(\frac{n}{D}, \frac{n}{DB} \log_{M/B} \frac{n}{DB}\right)\right)$. Nel caso reale (B grande) il secondo termine domina ed è = al bound di sorting: *permutare costa quanto ordinare*, cosa falsa nel modello RAM.

2 Warm-up del corso: Maximum Subarray e List Ranking

Questi due “warm-up” (cap. 2 e 4 della dispensa) non sono mai usciti come domanda teorica d’esame, ma sono parte del programma e servono a fissare i modelli (RAM vs I/O, cache-obliviousness, pointer jumping). Li includiamo per completezza: così non c’è nulla del corso che ti sia estraneo.

2.1 Maximum Subarray Sum

Richiami di teoria — il problema e le tre complessità

Dato $D[1, n]$ (con valori anche negativi), trovare il sottoarray contiguo $D[b_o, s_o]$ di somma massima. Quattro algoritmi, complessità decrescente:

- **Cubico** $\Theta(n^3)$: prova tutte le coppie (b, s) e ricalcola ogni somma da capo.
- **Quadratico** $\Theta(n^2)$: per ogni b , estende s accumulando la somma (riusa il calcolo precedente).
- **Lineare** $\Theta(n)$ — **Kadane**: una sola scansione, $O(1)$ spazio aggiuntivo.
- **Lineare** — **Prefix Sums**: $\max_s (P[s] - \min_{b \leq s} P[b - 1])$, con $P[i] = \sum_{j \leq i} D[j]$; calcola il minimo prefisso in modo incrementale.

Richiami di teoria — Kadane: correttezza e I/O

```
MaxSum ← -∞; TmpSum ← 0; b ← 1
for s = 1..n:
    TmpSum ← TmpSum + D[s]
    if MaxSum < TmpSum: MaxSum ← TmpSum; b_o ← b; s_o ← s
    if TmpSum < 0: TmpSum ← 0; b ← s+1
return (MaxSum, b_o, s_o)
```

Due proprietà strutturali dell’ottimo $D[b_o, s_o]$ (sono il cuore della correttezza):

- **Proprietà 1**: ogni prefisso $D[x, b_o - 1]$ ($x < b_o$) ha somma ≤ 0 — altrimenti si estenderebbe l’ottimo a sinistra aumentando la somma (assurdo). $\Rightarrow$ quando $TmpSum < 0$ si può ripartire da $s + 1$.
- **Proprietà 2**: ogni prefisso $D[b_o, y]$ ($y \leq s_o$) ha somma ≥ 0 — altrimenti si troncherebbe l’ottimo aumentando la somma. $\Rightarrow$ si aggiorna $MaxSum$ solo quando $TmpSum$ migliora.

Teorema 2.1. Kadane risolve il problema in $\Theta(n)$ tempo, $O(1)$ spazio, ed esegue n/B I/O, che è I/O-ottimo. Inoltre B non compare nel codice: l’algoritmo è **cache-oblivious**.

Perché è un buon esempio introduttivo: mostra che un algoritmo a singola scansione sequenziale è automaticamente I/O-ottimo e cache-oblivious — il principio di località spaziale del cap. 1 “gratis”.

2.2 List Ranking

Richiami di teoria — il problema e perché è interessante

Data una lista concatenata L di n elementi *sparsi su disco*, calcolare per ogni elemento x il suo $\text{rank}(x) = \text{numero di successori}$ (incluso sé). La scansione naïve è $O(n)$ in RAM ma $O(n)$ I/O in memoria esterna (ogni *next* può saltare su una pagina diversa): *catastrofico*. Serve un algoritmo I/O-efficiente.

Richiami di teoria — Pointer Jumping

Tecnica parallela: a ogni passo ogni nodo “salta” al successore del successore, *dimezzando* la distanza dalla fine.

```
init: se next[i]=NIL: rank[i]=1; else rank[i]=0, w[i]=1
while esistono nodi con next[i]≠NIL:
  for ogni i con next[i]≠NIL (in parallelo):
    rank[i] += w[next[i]]
    w[i]    += w[next[i]]
    next[i] = next[next[i]]
```

Teorema 2.2. *Pointer Jumping* risolve List Ranking in $O(\log n)$ passi paralleli ($\lceil \log_2 n \rceil$), ciascuno di $O(n)$ operazioni: $O(n \log n)$ lavoro sequenziale.

Limite in memoria esterna: ogni salto di puntatore può causare un I/O fault, quindi il pointer jumping diretto è I/O-inefficiente. Serve un approccio divide-and-conquer.

Richiami di teoria — algoritmo esterno divide-and-conquer

Idea: selezionare un *insieme indipendente* I (nessun nodo di I è successore di un altro di I), rimuoverlo aggiornando i pesi, risolvere ricorsivamente, reintegrare.

```
if n <= M: calcola rank in memoria interna; return
1. costruisci I casuale: ogni nodo entra con prob. 1/2 se il
   predecessore non è in I
2. per ogni v in I: w[prev(v)] += w[v]; collega prev(v)->next(v)
3. ListRanking(L \ I) // ricorsione
4. per ogni v in I: rank[v] = rank[prev(v)] + w[v] // reintegra
```

Teorema 2.3. L'algoritmo esegue $O(\text{sort}(n))$ I/O attesi, con $\text{sort}(n) = O\left(\frac{n}{B} \log_{M/B} \frac{n}{B}\right)$.

Sketch. L'insieme indipendente casuale ha taglia attesa $\geq n/4$, quindi la lista si riduce di un fattore costante per livello $\Rightarrow O(\log n)$ livelli. Ogni livello usa un *ordinamento* per riallineare i puntatori ai blocchi su disco ($O(\text{sort}(n))$ I/O); l'analisi ingenua dà $O(\log n \cdot \text{sort}(n))$, ma con tecniche di *compaction* si scende a $O(\text{sort}(n))$. $\square$

Morale: è l'esempio archetipico di “trasformare un problema con accessi puntatore sparsi (pessimi in I/O) in una sequenza di *sort/scan*”, il paradigma ricorrente dell'algoritmica in memoria esterna.

3 Ordinamento: tecniche, lower bound e varianti

3.1 Snow Plow: lunghezza attesa del run = $2M$

Richiami di teoria — Snow Plow (replacement selection)

La tecnica *Snow Plow* (Knuth) serve a generare i run iniziali del MergeSort producendo run più lunghi di M (in media $2M$), così da ridurre il numero di run e quindi le passate di fusione. Usa un min-heap $\mathcal{H}$ di taglia M in memoria interna e un buffer “non ordinato” $\mathcal{U}$.

```
build min-heap H dai primi M elementi;  $\mathcal{U} \leftarrow \emptyset$ 
while  $H \neq \emptyset$ :
    min  $\leftarrow$  extract-min(H); scrivi min nel run corrente
    next  $\leftarrow$  prossimo elemento dello stream
    if next < min: inserisci next in  $\mathcal{U}$  // non può stare in questo run
    else: inserisci next in H // estende il run corrente
quando H si svuota: il run finisce;  $\mathcal{U}$  diventa il nuovo H; nuova fase
```

Teorema 3.1 (Correttezza). Ogni fase produce un run ordinato.

Proof. Durante una fase il minimo estratto da $\mathcal{H}$ è non decrescente: un nuovo elemento entra in $\mathcal{H}$ solo se $\geq$ dell'ultimo minimo emesso, mentre se è più piccolo va in $\mathcal{U}$ (rinviato al run successivo). Quindi gli elementi scritti escono in ordine non decrescente: il run è ordinato. Inoltre tutti gli elementi presenti in $\mathcal{H}$ all'inizio della fase vengono scritti prima che la fase finisca. $\square$

Richiami di teoria — la lunghezza attesa

Teorema 3.2 (Lunghezza media del run). Su input con distribuzione casuale, una fase di Snow Plow produce in media un run di lunghezza $2M$. Più in generale, se ogni elemento letto finisce nel buffer $\mathcal{U}$ (cioè “è troppo piccolo”) con probabilità p , la lunghezza attesa del run è M/p .

Proof. Sia τ il numero totale di elementi processati in una fase (= lunghezza del run, perché ogni elemento estratto da $\mathcal{H}$ viene scritto). All'inizio $\mathcal{H}$ contiene M elementi. Per ciascuno degli elementi letti dallo stream durante la fase, esso finisce in $\mathcal{U}$ con probabilità p e resta in $\mathcal{H}$ (estendendo il run) con probabilità $1 - p$. La fase termina quando $\mathcal{H}$ si svuota e contemporaneamente $\mathcal{U}$ si è riempito fino a M (diventa il prossimo heap). Dunque il numero atteso di elementi finiti in $\mathcal{U}$ è M :

$$p \cdot \tau = M \implies \tau = \frac{M}{p}$$

Per $p = \frac{1}{2}$ (input casuale uniforme) si ha $\tau = 2M$. $\square$

Lettura “a flusso” equivalente. A ogni passo si scrive 1 elemento e se ne legge 1, che resta in $\mathcal{H}$ con prob. $1 - p$: la taglia di $\mathcal{H}$ cala in media di p per passo. Partendo da M , $\mathcal{H}$ si svuota dopo M/p passi: stessa conclusione.

Domanda d'esame — Snow Plow = $2M$ e varianti

Prove that the expected length of an ordered sequence produced by Snow Plow is $2M$. What is that expected length if the probability for an item to go in the “unsorted bucket” is $\frac{1}{4}$ instead of $\frac{1}{2}$? (Variante [240621]: probabilità $\frac{1}{8}$. Variante [201111]: probabilità di andare nell'heap = $\frac{3}{4}$.)

Soluzione svolta

La dimostrazione del caso $p = \frac{1}{2}$ è quella del Teorema 3.2: $p\tau = M \Rightarrow \tau = 2M$. La formula generale è $\tau = M/p$, dove p è la probabilità che un elemento letto vada nel buffer non ordinato $\mathcal{U}$. Quindi:

- $p = \frac{1}{4}$ [240115]: $\tau = M/(1/4) = 4M$.
- $p = \frac{1}{8}$ [240621]: $\tau = M/(1/8) = 8M$.
- probabilità di andare nell'heap = $\frac{3}{4}$ [201111]: allora $p = 1 - \frac{3}{4} = \frac{1}{4}$, quindi $\tau = 4M$.

Intuizione: più è raro che un elemento sia “troppo piccolo” (cioè p piccolo), più a lungo l'heap resta pieno e più lungo è il run; al limite $p \rightarrow 0$ (input già ordinato) il run diventa unico di lunghezza n .

3.2 Bounded QuickSort (profondità di stack $O(\log n)$)**Richiami di teoria**

La QuickSort ricorsiva classica può avere stack di ricorsione profondo $\Theta(n)$ nel caso peggiore (partizioni sbilanciate). La Bounded QuickSort elimina la ricorsione sul sottoarray più grande trasformandola in iterazione (tail-recursion elimination) e ricorre solo sul sottoarray più piccolo. Così ogni chiamata ricorsiva lavora su $\leq n/2$ elementi e la profondità dello stack è $\leq \log_2 n$ garantito.

```
BoundedQuickSort(S, i, j):
  while i < j:
    p ← Partition(S, i, j)    // p = posizione finale del pivot
    if (p - i) < (j - p):    // parte sinistra più piccola
      BoundedQuickSort(S, i, p-1) // ricorri sulla piccola
      i ← p + 1                // itera sulla grande (destra)
    else:
      BoundedQuickSort(S, p+1, j)
      j ← p - 1
```

Commento. La correttezza è quella della QuickSort (il pivot è in posizione finale, i due lati vengono ordinati). La novità è di sistema: ricorrendo sempre sulla metà $\leq n/2$, dopo d chiamate annidate la taglia è $\leq n/2^d$, quindi $d \leq \log_2 n$: lo spazio di stack è $O(\log n)$ nel caso peggiore, mentre il tempo resta $O(n \log n)$ atteso. È la versione “robusta” da usare in produzione.

Domanda d'esame — Bounded QuickSort

Write and comment the pseudo-code of the Bounded Quicksort.

Soluzione svolta

Lo pseudocodice è quello del riquadro sopra. I punti da scrivere/commentare all'esame: (1) si partiziona attorno a un pivot e si ottiene la sua posizione finale p ; (2) si confrontano le taglie dei due lati e si *ricorre sul più piccolo*, aggiornando i o j per *iterare* sul più grande; (3) *garanzia*: profondità di stack $\leq \log_2 n$ perché la chiamata ricorsiva è sempre su $\leq n/2$ elementi, mentre la parte grande non consuma stack. Tempo $O(n \log n)$ atteso con pivot casuale (o garantito con pivot = mediana di un campione).

3.3 Lower bound di ordinamento (modello a due livelli)**Richiami di teoria — argomento ad albero di decisione**

Baseline RAM. Un algoritmo *comparison-based* è un albero binario di decisione con $n!$ foglie (una per permutazione); l'altezza è $\geq \lceil \log_2 n! \rceil = \Omega(n \log n)$ (Stirling).

Modello a due livelli. Ora ogni nodo dell'albero è *un I/O*. Quanti esiti distinti può produrre un I/O? Leggendo una pagina di B nuovi elementi e “mescolandoli” con gli $M - B$ già in memoria, si generano al più $\binom{M}{B} B!$ ordinamenti relativi distinti. Dopo t I/O le foglie raggiungibili sono $\leq \left(\binom{M}{B} B!\right)^t$. Distinguendo i $\frac{n}{B}$ I/O che leggono pagine “vergini”:

$$\left(\binom{M}{B} B!\right)^{t - \frac{n}{B}} \cdot \left(n \binom{M}{B}\right)^{\frac{n}{B}} \geq n!.$$

Risolvendo per t con Stirling si ottiene il Teorema 1.2:

$$t = \Omega\left(\frac{n}{B} \log_{M/B} \frac{n}{B}\right) \quad (D = 1).$$

Quindi il Multiway MergeSort è *asintoticamente ottimo*.

3.4 Ordinamento di stringhe: lower bound, RadixSort, Multikey QuickSort**Richiami di teoria — prefisso distinguente e lower bound**

Sia S un insieme di n stringhe, lunghezza totale N , alfabeto di taglia σ . Ordinare con MergeSort/QuickSort confrontando stringhe intere costa $O(N \log n)$, che è *sub-ottimo*.

Per ogni stringa s , sia d_s la lunghezza del *più corto prefisso* che distingue s da tutte le altre; sia $d = \sum_s d_s$ il *prefisso distinguente totale*.

Lemma 3.3 (Lower bound per string sorting). *Ogni algoritmo che ordina S esegue $\Omega(d + n \log n)$ confronti di caratteri.*

Interpretazione: d è il costo minimo per “distinguere” tutte le stringhe (bisogna leggere almeno il loro prefisso distinguente); $n \log n$ è il costo dell'albero di decisione per metterle in ordine. MergeSort/QuickSort fanno $O(N \log n)$: sprecano un fattore $\log n$ quando $d \ll N$.

Richiami di teoria — RadixSort MSD e LSD

MSD-first (most significant digit). Distribuisce ricorsivamente le stringhe in σ bucket secondo il carattere corrente, partendo dal primo: *equivale a costruire una trie*. *Teorema*: con array di σ posizioni per nodo, *tempo e spazio* $O(d\sigma)$ (ogni stringa percorre un cammino di d_s nodi; in totale $\leq d$ nodi, ognuno $O(\sigma)$); *con hashing nei nodi*, $O(d \log \sigma)$ atteso e $O(d)$ spazio.

LSD-first (least significant digit). *Ordina ripetutamente per cifra, dalla meno alla più significativa, usando a ogni passo un ordinamento stabile* (CountingSort). Assumiamo n stringhe della stessa lunghezza L .

Teorema 3.4 (Complessità e correttezza dell'LSD RadixSort). *LSD RadixSort ordina n stringhe di lunghezza L su alfabeto σ in $O(L(n + \sigma)) = O(N + L\sigma)$ tempo e $O(N)$ spazio. È corretto.*

Proof. **Complessità.** Si eseguono L fasi; ogni fase è un CountingSort sulla cifra i con costo $O(n + \sigma)$ (conteggio dei σ simboli + ricollocazione degli n elementi). Totale $O(L(n + \sigma))$. Con cifre da r bit alla volta ($r = \Theta(\log n)$) il costo scende a $O(bn / \log n)$ per stringhe di b bit.

Correttezza (per induzione sul numero di fasi, sfruttando la stabilità). Invariante: dopo la fase che ordina sulla cifra i (contando da destra), le stringhe sono ordinate secondo le ultime i cifre. Base $i = 0$: banale. Passo: assumiamo l'ordine sulle ultime $i - 1$ cifre; la fase i ordina (stabilmente) per la i -esima cifra da destra. Prese due stringhe α, β :

- se differiscono nella cifra i , il CountingSort le mette nell'ordine giusto secondo quella cifra (cifra più significativa fra le ultime i): corretto;
- se hanno la stessa cifra i , la *stabilità* preserva l'ordine relativo che già avevano per induzione (ultime $i - 1$ cifre): corretto.

Dopo L fasi l'ordine è completo. È essenziale che ogni fase sia *stabile*: con un ordinamento non stabile l'invariante si romperebbe nei pareggi. $\square$

Domanda d'esame — LSD RadixSort

Sketch the LSD radix-sort algorithm, state its time complexity, and prove that the algorithm is correct (case of binary strings).

Soluzione svolta

Pseudocodice: per $i = L$ gi'ù fino a 1, esegui un CountingSort stabile sulla cifra i -esima (la più a destra per prima). Complessità $O(L(n + \sigma))$ tempo, $O(N)$ spazio (Teorema 3.4). Per stringhe binarie ($\sigma = 2$) ogni fase è una distribuzione 0/1 stabile, e con $L = b$ bit il costo è $O(bn)$ (oppure $O(bn / \log n)$ raggruppando $\Theta(\log n)$ bit). **Correttezza**: la dimostrazione induttiva sopra — dopo aver ordinato sulle ultime i cifre l'array è ordinato per quel suffisso, grazie alla stabilità del CountingSort sui pareggi. Dopo b fasi le stringhe sono completamente ordinate.

Richiami di teoria — Multikey QuickSort

Generalizza la QuickSort a 3 vie agli array di stringhe, confrontando un carattere alla volta (la posizione i).

MultikeyQS(R, i):

```

if |R| <= 1: return R
scegli pivot p in R (casuale); c = p[i]
R< = {s in R : s[i] < c}; R= = {s : s[i]=c}; R> = {s : s[i] > c}
return MultikeyQS(R<, i) ++ MultikeyQS(R=, i+1) ++ MultikeyQS(R>,

```

i)

Sul ramo $R_=_$ si avanza alla posizione $i + 1$ (i caratteri fino a i sono uguali); sui rami $R_<, R_>$ si resta su i . È l'analogo dinamico del *Ternary Search Tree*.

Teorema 3.5 (Ottimalità di Multikey QuickSort). *Multikey QuickSort esegue $O(d + n \log n)$ confronti di caratteri attesi, ottimo per il Lemma 3.3.*

Idea. Per ogni stringa s : ogni volta che finisce in $R_<$ o $R_>$, viene confrontato un carattere $s[i]$ e s scende in un sottoinsieme di taglia $\leq \alpha|R|$ (come nella QuickSort classica): il contributo totale di questi confronti è $O(\log n)$ atteso per stringa, cioè $O(n \log n)$. Ogni volta che finisce in $R_=_$ si confronta $s[i]$ e si avanza i : ciò accade al più d_s volte (fino a quando s è distinta da tutte). Sommando: $O(n \log n + \sum_s d_s) = O(n \log n + d)$. $\square$

Domanda d'esame — Multikey QS + lower bound

Write the pseudocode of Multikey Quicksort and prove its time complexity; also state and prove the lower bound for sorting n strings of total length N .

Soluzione svolta

Lower bound: Lemma 3.3, $\Omega(d + n \log n)$ confronti di caratteri: d per distinguere le stringhe (occorre leggerne i prefissi distinguenti), $n \log n$ per ordinarle (albero di decisione con $n!$ foglie).

Multikey QS: pseudocodice del riquadro; complessità attesa $O(d + n \log n)$ (Teorema 3.5), che *eguaglia* il lower bound: l'algoritmo è quindi ottimo nel modello a confronti. Worst-case ottimo prendendo come pivot la mediana di un campione.

4 Random Sampling e Reservoir Sampling

Richiami di teoria — il problema

Estrarre un campione di m elementi *uniformemente a caso* (ogni elemento con probabilità m/n) da una sequenza di n elementi. Tre scenari:

- **Disco, n noto**: genera m posizioni distinte casuali in $[1, n]$ e leggile.
- **Streaming, n noto**: scorri la sequenza una volta e decidi al volo.
- **Streaming, n ignoto**: è il caso interessante — *Reservoir Sampling*.

Nello streaming si fa *una sola passata*, gli elementi arrivano uno alla volta e non si possono ri-leggere; n si scopre solo alla fine (o mai).

4.1 Reservoir Sampling (n ignoto)

Richiami di teoria — algoritmo

Si tiene un “serbatoio” (reservoir) $R[1..m]$ con i primi m elementi; per ogni elemento successivo si decide se rimpiazzarne uno a caso.

```
ReservoirSampling(stream, m):  
  for i = 1 to m:  $R[i] \leftarrow \text{stream}[i]$     // riempi il reservoir  
   $i \leftarrow m$   
  for each successivo elemento x ( $i = m+1, m+2, \dots$ ):  
     $j \leftarrow$  intero casuale uniforme in  $[1, i]$   
    if  $j \leq m$ :  $R[j] \leftarrow x$     // x entra nel campione, sfratta  $R[j]$   
    // altrimenti x viene scartato  
  return R
```

Costo: $O(n)$ tempo (un passo per elemento), $O(m)$ spazio. Una sola passata.

Richiami di teoria — correttezza

Teorema 4.1 (Uniformità del Reservoir Sampling). *Al termine, dopo aver visto n elementi, ogni elemento della sequenza si trova nel reservoir con probabilità esattamente m/n .*

Proof. Fissiamo l'elemento arrivato in posizione t e calcoliamo la probabilità che, dopo n elementi, sia ancora in R .

Probabilità che t entri. Se $t \leq m$ entra con certezza. Se $t > m$, entra quando estraiamo $j \leq m$ con j uniforme in $[1, t]$: probabilità m/t . (Per $t \leq m$ la formula m/t darebbe ≥ 1 , ma in quel caso la probabilità è 1; trattiamo direttamente il caso generico t e mostriamo che il prodotto telescopica a m/n .)

Probabilità di sopravvivere a ogni passo successivo $i > t$. All'arrivo dell' i -esimo elemento si estrae $j \in [1, i]$; l'elemento t è rimpiazzato solo se $j \leq m$ e j coincide con la sua cella, evento di probabilità $\frac{1}{i}$. Quindi t sopravvive a quel passo con probabilità $1 - \frac{1}{i} = \frac{i-1}{i}$.

Prodotto. Per $t \geq m$ (il caso $t < m$ è analogo, l'ingresso è certo):

$$\Pr[t \in R \text{ alla fine}] = \underbrace{\frac{m}{t}}_{\text{entra}} \cdot \prod_{i=t+1}^n \underbrace{\frac{i-1}{i}}_{\text{sopravvive}} = \frac{m}{t} \cdot \frac{t}{t+1} \cdot \frac{t+1}{t+2} \cdots \frac{n-1}{n} = \frac{m}{t} \cdot \frac{t}{n} = \frac{m}{n}.$$

Il prodotto telescopico cancella tutto tranne t al numeratore e n al denominatore. Quindi ogni elemento è nel campione con probabilità m/n , indipendente da t : il campionamento è uniforme. $\square$

Domanda d'esame — Reservoir Sampling

Write the pseudocode of Reservoir Sampling (unknown n) and prove its correctness (each item is sampled with probability m/n).

Variante [231106] (midterm): the sampling algorithm that knows n and draws m items uniformly; prove correctness for the case $m = 1$.

Soluzione svolta

Pseudocodice + prova: come nei riquadri sopra; il cuore è il prodotto telescopico $\frac{m}{t} \prod_{i=t+1}^n \frac{i-1}{i} = \frac{m}{n}$ (Teorema 4.1).

Variante con n noto, $m = 1$. Si mantiene un solo elemento r ; all'arrivo dell' i -esimo elemento lo si sostituisce con probabilità $1/i$. Allora l'elemento t è quello finale con probabilità

$$\frac{1}{t} \prod_{i=t+1}^n \left(1 - \frac{1}{i}\right) = \frac{1}{t} \prod_{i=t+1}^n \frac{i-1}{i} = \frac{1}{t} \cdot \frac{t}{n} = \frac{1}{n},$$

cioè m/n con $m = 1$: ogni elemento è scelto con probabilità $1/n$, uniforme. (Con n noto si può anche, equivalentemente, estrarre direttamente una posizione casuale in $[1, n]$.)

5 Intersezione di insiemi ordinati

Richiami di teoria — il problema e gli approcci base

Date due sequenze *ordinate* di interi $A = a_1 \cdots a_n$ e $B = b_1 \cdots b_m$ con $m \leq n$, calcolare $A \cap B$. È il cuore dei motori di ricerca (intersezione di liste di posting).

- **Merge** (scansione a due puntatori): $O(m + n)$ passi, $O((m + n)/B)$ I/O, cache-oblivious. Lower bound $\Omega(\min(n, m))$ (ogni elemento di B va comunque toccato).
- **Binary search**: cerca ogni $b \in B$ in A : $O(m \log n)$. Conviene al merge quando $m = o(n / \log n)$ (cioè B molto più piccolo di A).

5.1 Mutual Partitioning

Richiami di teoria

Si sceglie la mediana di B e la si cerca in A , spezzando ricorsivamente entrambe le liste.

```
Intersect(A[1,n], B[1,m]):  
  if m = 0: return  
  p ← B[floor(m/2)] // mediana di B  
  binary search p in A: trova j t.c. A[j] ≤ p < A[j+1]  
  if A[j] = p: stampa p  
  Intersect(A[1,j], B[1, m/2 - 1])  
  Intersect(A[j+1,n], B[m/2 + 1, m])
```

Teorema 5.1 (Complessità di Mutual Partitioning). *Esegue $O(m(1 + \log \frac{n}{m}))$ confronti, ottimo nel modello a confronti.*

Proof. Ricorrenza $T(n, m) = O(\log n) + 2T(n/2, m/2)$ con base $T(n, 0) = O(1)$: ci sono $\Theta(\log m)$ livelli di ricorsione su B (si dimezza m), e a ogni livello la somma delle binary search sulle porzioni di A è $O(m \log \frac{n}{m})$ (concavità del log / Jensen). **Ottimalità**: esistono $\binom{n}{m}$ possibili posizioni relative, quindi un albero di decisione ha $\geq \binom{n}{m}$ foglie e profondità $\geq \log \binom{n}{m} = \Omega(m \log \frac{n}{m})$. $\square$

5.2 Doubling / Exponential (Gallop) Search

Richiami di teoria

Per ogni $b_j \in B$ si “galoppa” in A a salti 1, 2, 4, 8, ... dall’ultima posizione raggiunta, finché si supera b_j ; poi binary search nell’ultimo intervallo $[2^{k-1}, 2^k]$.

```
i ← 0  
for j = 1..m:  
  k ← 0  
  while (i + 2^k ≤ n) and (B[j] > A[i + 2^k]): k++  
  i' ← binary search di B[j] in A[i + 2^(k-1) + 1, min(i + 2^k, n)]  
  if A[i'] = B[j]: stampa B[j]  
  i ← i'
```

Teorema 5.2 (Complessità del Doubling Search). *Esegue $O(m(1 + \log \frac{n}{m}))$ passi, ottimo.*

Proof. Sia $\Delta_j = \min(2^{k_j}, n)$ la taglia del sottovettore di A su cui si fa la binary search per b_j ; per la natura del raddoppio $\Delta_j \leq 2(i_j - i_{j-1})$, dove i_j è la posizione finale. Il costo per b_j è $O(1 + \log \Delta_j)$. Sommando e usando la concavità del log (Jensen):

$$\sum_{j=1}^m O(1 + \log \Delta_j) = O\left(m + \log \prod_j \Delta_j\right) \leq O\left(m + m \log \frac{\sum_j \Delta_j}{m}\right) = O\left(m \log \frac{n}{m}\right),$$

perché $\sum_j \Delta_j = O(n)$ (gli intervalli coprono A a tratti, con sovrapposizione costante).

Vantaggio sul mutual partitioning: accesso *iterativo* ad A (niente ricorsione) $\Rightarrow$ migliore località di cache. $\square$

Domanda d'esame — Doubling intersection ; Mutual partition

Describe the doubling algorithm to compute the intersection of two sorted lists L_1, L_2 of lengths n and m ; state and prove its time complexity. (Variante [240711]: describe the mutual-partition algorithm.)

Soluzione svolta

Doubling: pseudocodice del riquadro; complessità $O(m \log \frac{n}{m})$ (Teorema 5.2), con la prova via $\Delta_j \leq 2(i_j - i_{j-1})$ e Jensen. **Mutual partition:** Teorema 5.1, stessa complessità $O(m \log \frac{n}{m})$, ottima perché $\log \binom{n}{m} = \Omega(m \log \frac{n}{m})$. Entrambi battono il merge $O(n + m)$ quando $m \ll n$ e la binary search ripetuta $O(m \log n)$ quando $m \approx n$.

5.3 2-Level Intersection (modello a due livelli)

Richiami di teoria — algoritmo a due livelli

Si sfrutta la struttura a blocchi del disco per ridurre gli I/O.

- **Preprocessing.** Si divide A (lungo n) in $h = n/L$ blocchi $A_1, \dots, A_h$ di taglia L ; si costruisce un array ausiliario $A' = (A_1[1], A_2[1], \dots, A_h[1])$ con i primi elementi di ciascun blocco (taglia $h = n/L$).
- **Fase 1.** Si fa il merge di A' con B : per ogni $b \in B$ si individua l'unico blocco A_i che può contenerlo. Costo $O(n/L + m)$.
- **Fase 2.** Per ogni blocco A_i "selezionato" (con dentro candidati di B) si calcola $A_i \cap B_i$ con un merge locale. Costo $O(L + |B_i|)$ per blocco, $O(mL + m)$ in totale (al più m blocchi non vuoti).

Teorema 5.3 (Complessità 2-Level Intersection). *Tempo* $O(n/L + mL)$ e *I/O* $O(\frac{n}{LB} + \frac{m}{B} + m)$. Con la scelta ottima $L = \sqrt{n/m}$ il tempo è $O(\sqrt{nm})$.

Proof. **Tempo.** Fase 1 = $O(n/L + m)$ (scansione di A' e B). Fase 2 = $\sum_i (L + |B_i|) = O(mL + m)$ (ogni elemento di B cade in un blocco; al più m blocchi non vuoti, ciascuno scandito in $O(L)$). Totale $O(n/L + mL)$. Minimizzando $f(L) = n/L + mL$: $f'(L) = -n/L^2 + m = 0 \Rightarrow L = \sqrt{n/m}$, da cui $f(\sqrt{n/m}) = 2\sqrt{nm} = O(\sqrt{nm})$.

I/O. La Fase 1 scorre sequenzialmente A' (taglia n/L) e B (taglia m): $O(\frac{n}{LB} + \frac{m}{B})$ I/O. La Fase 2 accede ai blocchi selezionati, ognuno è una pagina: $O(m)$ I/O. **Bilanciamento:** $O(\sqrt{nm})$ batte sia $O(n + m)$ (quando $m \ll n$) sia $O(m \log n)$ (quando $m \approx n$). $\square$

Domanda d'esame — 2-Level Intersection

Describe the two-level intersection algorithm and show/prove its time complexity (in the RAM model). ([241111], [250715]: “show and prove the time complexity”.)

Soluzione svolta

Descrizione in tre passi (preprocessing $\rightarrow$ Fase 1 merge di A' con $B \rightarrow$ Fase 2 merge locale nei blocchi selezionati) e prova del Teorema 5.3: tempo $O(n/L + mL)$, minimizzato a $O(\sqrt{nm})$ con $L = \sqrt{n/m}$, I/O $O(\frac{n}{LB} + \frac{m}{B} + m)$. Il punto da non dimenticare: L è un parametro libero che bilancia il costo della Fase 1 (scansione dei rappresentanti, $\propto n/L$) con quello della Fase 2 (scansione dei blocchi, $\propto mL$).

5.4 Intersezione di liste compresse con Elias–Fano**Richiami di teoria**

Se A e B sono memorizzate compresse con **Elias–Fano** (cap. 10), che offre le primitive **ACCESS**(j) in $O(1)$ e **NEXTGEQ**(x) (il più piccolo elemento $\geq x$) in tempo efficiente, si può **intersecare senza decomprimere, alternando NEXTGEQ fra le due liste** (galloping su strutture compresse).

```
i ← 1; x ← A.Access(1)
loop:
  y ← B.NextGEQ(x)
  if y = x: stampa x; x ← A.NextGEQ(y+1)
  else:    x ← A.NextGEQ(y)
  // scambia i ruoli di A e B e ripeti finché una lista finisce
```

Si paga $O(\text{NEXTGEQ})$ per ogni avanzamento; il numero di avanzamenti è $O(m \log \frac{n}{m})$ come nel galloping, ma su rappresentazione compressa (meno spazio, meno I/O).

Domanda d'esame — intersezione su Elias–Fano

Write the pseudocode to intersect two Elias–Fano-encoded sorted lists using the ACCESS/GEQ primitives, and analyze the time complexity.

Soluzione svolta

Pseudocodice del riquadro: si avanza alternativamente con NEXTGEQ sulla lista “indietro”. Ogni operazione ACCESS/NEXTGEQ su Elias–Fano costa $O(1)$ ammortizzato (grazie alla Select sul bit-vector dei bucket alti); il numero di passi è $O(m \log \frac{n}{m})$ come nel doubling search, quindi il costo totale è $O(m \log \frac{n}{m})$ ma con il vantaggio di operare **in spazio compresso** ($\approx n \log(u/n) + 2n$ bit per lista) e quindi con molti meno trasferimenti dal disco.

6 Dizionari: hashing, perfect hashing, cuckoo, Bloom, Skip List

6.1 Hashing universale

Richiami di teoria — chaining e hashing uniforme

Una hash table con *chaining* memorizza in $T[h(k)]$ una lista delle chiavi con quel valore di hash. Con *simple uniform hashing* (ogni chiave equiprobabile su m slot, indipendente):

Teorema 6.1 (Costo della ricerca con chaining). *Sotto hashing uniforme, $\text{SEARCH}(k)$ costa $\Theta(1 + \alpha)$ atteso, con $\alpha = n/m$ fattore di carico.*

La debolezza: nessuna singola funzione h è buona per ogni input (un avversario può mandare tutte le chiavi sullo stesso slot). Soluzione: scegliere h a caso da una famiglia con buone garanzie — le *classi universali*.

Richiami di teoria — definizione ed esempio

Definizione 6.2 (Classe universale). Una collezione finita $\mathcal{H}$ di funzioni $h : U \rightarrow \{0, \dots, m-1\}$ è *universale* se per ogni coppia di chiavi distinte $x \neq y$:

$$|\{h \in \mathcal{H} : h(x) = h(y)\}| \leq \frac{|\mathcal{H}|}{m},$$

ovvero, scegliendo h a caso in $\mathcal{H}$, $\Pr[h(x) = h(y)] \leq 1/m$.

Teorema 6.3 (Chaining con classe universale). *Con h presa a caso da una classe universale, la lunghezza attesa di una lista è $\leq 1 + \alpha$ per qualunque input (nessuna assunzione sulla distribuzione delle chiavi).*

Proof. Per x fissato sia N_x il numero di chiavi nella sua lista e $I_{xy} = \mathbf{1}[h(x) = h(y)]$. Per universalità $E[I_{xy}] \leq 1/m$, quindi $E[N_x] = \sum_{y \neq x} E[I_{xy}] \leq (n-1)/m < \alpha$, più la chiave x stessa. $\square$

Esempio (classe del prodotto scalare). Sia m primo e si vedano le chiavi in base m , $k = (k_0, k_1, \dots, k_{r-1})$. Per ogni vettore $a = (a_0, \dots, a_{r-1})$ con $a_i \in \{0, \dots, m-1\}$:

$$h_a(k) = \left(\sum_{i=0}^{r-1} a_i k_i \right) \bmod m.$$

La famiglia $\mathcal{H} = \{h_a\}$ ha $|\mathcal{H}| = m^r$ funzioni.

Teorema 6.4 (Universalità). *La classe $\mathcal{H} = \{h_a\}$ con m primo è universale.*

Proof. Siano $x \neq y$ distinte: differiscono in almeno una cifra, sia d tale che $x_d \neq y_d$. La collisione $h_a(x) = h_a(y)$ equivale a

$$\sum_i a_i (x_i - y_i) \equiv 0 \pmod{m} \iff a_d (x_d - y_d) \equiv -\sum_{i \neq d} a_i (x_i - y_i) \pmod{m}.$$

Poiché m è primo e $(x_d - y_d) \not\equiv 0$, l'elemento $(x_d - y_d)$ è invertibile modulo m . Quindi, fissati arbitrariamente gli a_i per $i \neq d$ (sono m^{r-1} scelte), esiste esattamente un valore di a_d che soddisfa l'uguaglianza. Le funzioni che fanno collidere x, y sono dunque m^{r-1} , e

$$\Pr[h_a(x) = h_a(y)] = \frac{m^{r-1}}{m^r} = \frac{1}{m}.$$

La condizione della Definizione 6.2 è soddisfatta con uguaglianza: $\mathcal{H}$ è universale. $\square$

Alternativa classica (utile da citare): $h_{a,b}(k) = ((ak + b) \bmod p) \bmod m$ con p primo $> |U|$, $a \in \{1, \dots, p-1\}$, $b \in \{0, \dots, p-1\}$: anch'essa universale con lo stesso argomento (per $x \neq y$ la mappa $(a, b) \mapsto ((ax + b) \bmod p, (ay + b) \bmod p)$ è una biiezione).

Domanda d'esame — Hashing universale

Define what a class of universal hash functions is; provide an example and prove its universality. ([231106]: show that with chaining it achieves $O(1)$ expected chain length when $m = O(n)$.)

Soluzione svolta

Definizione: 6.2. Esempio + prova: la classe del prodotto scalare $h_a(k) = (\sum a_i k_i) \bmod m$ con m primo, Teorema 6.4: per $x \neq y$ che differiscono nella cifra d , c'è una sola scelta di a_d che provoca collisione, da cui $\Pr[\text{coll}] = m^{r-1}/m^r = 1/m$. $O(1)$ con $m = O(n)$: per il Teorema 6.3 la lista attesa è $\leq 1 + \alpha = 1 + n/m$; se $m = \Theta(n)$ allora $\alpha = O(1)$ e quindi $\text{SEARCH} = O(1)$ atteso, per qualunque input.

6.2 Perfect Hashing (statico)

Richiami di teoria — schema FKS a due livelli

Definizione 6.5. h è **perfetta** per un insieme statico S di n chiavi se **non causa collisioni** fra esse.

Schema a due livelli: tabella primaria $T_1[0, n-1]$ con h_1 universale; per ogni slot j con n_j chiavi, una tabella secondaria $T_{2,j}$ di taglia $m_j = n_j^2$ con la sua h_2 universale.

Teorema 6.6. Mettere q chiavi in una tabella di taglia $s = q^2$ con hashing universale dà probabilità di collisione $< \frac{1}{2}$.

Proof. Le coppie sono $\binom{q}{2} < q^2/2$; ognuna collide con prob. $\leq 1/s = 1/q^2$; il numero atteso di collisioni è $< \frac{q^2}{2} \cdot \frac{1}{q^2} = \frac{1}{2}$; per Markov $\Pr[\geq 1 \text{ collisione}] < \frac{1}{2}$. $\square$

Teorema 6.7. Con T_1 di taglia n , $E[\sum_j n_j^2] < 2n$: lo spazio secondario totale atteso è $< 2n$.

Conseguenza: **ricerca in $O(1)$ worst case** (due lookup), **spazio $O(n)$ atteso**. È la base delle *Minimal Ordered Perfect Hash Functions* (MOPHF) usate spesso negli esercizi pratici.

6.3 Cuckoo Hashing

Richiami di teoria

Due funzioni h_1, h_2 : ogni chiave k sta in $T[h_1(k)]$ oppure $T[h_2(k)]$. **Ricerca e cancellazione $O(1)$ worst case** (due posizioni da controllare). Inserimento: si mette k in una delle due celle; se occupata, si "sfratta" l'inquilino, che va nella sua cella alternativa, a cascata; se il processo **cicla per più di m passi si fa un rehash**.

Grafo cuckoo: nodi = celle, archi = chiavi (arco da $T[h_1(k)]$ a $T[h_2(k)]$). **L'inserimento entra in loop se e solo se il grafo contiene un ciclo**.

Teorema 6.8 (Lunghezza dei cammini). Per ogni coppia di celle i, j e costante $c > 1$, se la tabella ha $r \geq 2cn$ posizioni (con n chiavi inserite), **la probabilità che esista un cammino da i**

a j di lunghezza $L \geq 1$ nel grafo cuckoo è $\leq c^{-L}/r$.

Corollario 6.9. Con c costante (es. $c = 3$, $r \geq 6n$), la probabilità di un ciclo (fallimento) è $\leq \frac{1}{c-1} \leq \frac{1}{2}$; il numero atteso di rehash è costante e l'inserimento è $\Theta(1)$ ammortizzato atteso.

Domanda d'esame — Cuckoo path theorem

State and prove the main theorem on the probability that the shortest path between positions i, j in the cuckoo graph has length L (form c^{-L}/r , with $r \geq 2cn$); comment its impact on the load factor.

Soluzione svolta

Enunciato: Teorema 6.8, $\Pr[\exists \text{ cammino } i \rightarrow j \text{ di lung. } L] \leq c^{-L}/r$. **Prova (induzione su L).** Base $L = 1$: c'è un arco diretto $i \rightarrow j$ solo se qualche chiave k ha $\{h_1(k), h_2(k)\} = \{i, j\}$; per ciascuna delle n chiavi ciò accade con prob. $\leq 2/r^2$ (le due funzioni colpiscono le due celle), quindi $\Pr \leq n \cdot 2/r^2 = 2n/r^2 \leq \frac{1}{c} \cdot \frac{1}{r} = c^{-1}/r$ usando $r \geq 2cn$. Passo: un cammino di lunghezza L da i a j è un cammino di lunghezza $L - 1$ da i a un nodo intermedio ℓ seguito da un arco $\ell \rightarrow j$:

$$\Pr[i \rightarrow j, \text{ lung. } L] \leq \sum_{\ell} \Pr[i \rightarrow \ell, L-1] \cdot \Pr[\ell \rightarrow j, 1] \leq \sum_{\ell} \frac{c^{-(L-1)}}{r} \cdot \frac{2n}{r^2} = r \cdot \frac{c^{-(L-1)}}{r} \cdot \frac{1}{cr} = \frac{c^{-L}}{r}.$$

Impatto sul load factor. La probabilità complessiva che i, j siano connessi è $\sum_{L \geq 1} c^{-L}/r = \frac{1}{r(c-1)}$; sommando sulle coppie/celle si ottiene prob. di ciclo $\leq \frac{1}{c-1}$. Per tenerla $\leq \frac{1}{2}$ serve $c \geq 3$, cioè $r \geq 6n$: il load factor $\alpha = n/r \leq 1/6$. Il cuckoo hashing classico (2 tabelle) garantisce $O(1)$ worst case in ricerca solo a fattori di carico bassi; per salire (oltre 50%) servono più funzioni o bucket più grandi.

6.4 Bloom Filter

Richiami di teoria

Bit-vector $B[0, m-1]$ inizializzato a 0 e r funzioni hash (universali) $h_1, \dots, h_r$.

- **Insert(k):** poni $B[h_i(k)] = 1$ per ogni i . Costo $O(r)$.
- **Search(y):** rispondi “presente” sse $B[h_i(y)] = 1$ per ogni i .
- **No cancellazione** (non si può azzerare un bit senza rischiare altri).

Correttezza unilaterale: nessun *falso negativo* (se k è stato inserito, tutti i suoi bit sono 1); **possibili falsi positivi** (i bit di y potrebbero essere stati messi a 1 da altre chiavi).

Richiami di teoria — probabilità di errore

Dopo aver inserito n chiavi (cioè rn “accensioni” di bit), la probabilità che un dato bit sia ancora 0 è

$$p_0 = \left(1 - \frac{1}{m}\right)^{rn} \approx e^{-rn/m}.$$

Un falso positivo richiede che tutti gli r bit di y siano 1:

$$\Pr[\text{falso positivo}] \approx (1 - p_0)^r = (1 - e^{-rn/m})^r.$$

Minimizzando rispetto a r (a m/n fissato) si trova l'ottimo

$$r^* = \frac{m}{n} \ln 2, \quad p_0 = \frac{1}{2}, \quad \Pr[\text{falso positivo}] \approx \left(\frac{1}{2}\right)^{r^*} = 2^{-(m/n) \ln 2}$$

Interpretazione: all'ottimo metà dei bit sono a 1 (massima entropia del filtro), e l'errore cala esponenzialmente in m/n (bit per chiave). Es. $m/n = 10$: $r^* \approx 7$, errore $\approx 0.8\%$.

Domanda d'esame — Bloom filter

Define a Bloom filter; compute the probability that a bit position is 0 (with the optimal number of hash functions) and derive/prove the false-positive error probability.

Soluzione svolta

Probabilità di un bit a 0: ogni accensione manca un bit fissato con prob. $1 - 1/m$; con rn accensioni indipendenti $p_0 = (1 - 1/m)^{rn} \approx e^{-rn/m}$. **Falso positivo:** servono tutti gli r bit di y a 1, prob. $(1 - p_0)^r$. **Ottimo:** derivando $(1 - e^{-rn/m})^r$ in r si ottiene $r^* = (m/n) \ln 2$, per cui $e^{-r^*n/m} = e^{-\ln 2} = \frac{1}{2}$, cioè $p_0 = \frac{1}{2}$, e l'errore vale $(\frac{1}{2})^{r^*} = 2^{-(m/n) \ln 2}$. Quindi all'ottimo la probabilità che un bit sia 0 è esattamente $\frac{1}{2}$ [201111], e l'errore decresce esponenzialmente nel numero di bit per chiave.

6.5 Skip List

Richiami di teoria

(Nota: questo risultato è richiesto agli esami ma non sviluppato nella dispensa; lo riportiamo nella forma standard, allineata al corso.) Una skip list memorizza una lista ordinata su più livelli: ogni elemento del livello ℓ è promosso al livello $\ell + 1$ con probabilità $\frac{1}{2}$, indipendentemente. La ricerca scende da sinistra-alto verso destra-basso.

Teorema 6.10 (Altezza attesa). Una skip list con n elementi ha altezza $O(\log n)$ con alta probabilità, e ricerca/inserimento in $O(\log n)$ atteso.

Proof. La probabilità che un dato elemento raggiunga il livello ℓ è $2^{-\ell}$ (deve essere “testa” ℓ volte). Il numero atteso di elementi al livello ℓ è $n 2^{-\ell}$. Per $\ell = c \log_2 n$ il numero atteso è $n \cdot n^{-c} = n^{1-c}$; per $c = 2$ vale $1/n$, quindi per l'union bound la probabilità che qualche elemento superi il livello $2 \log_2 n$ è $\leq n \cdot 2^{-2 \log_2 n} = 1/n \rightarrow 0$: l'altezza è $O(\log n)$ w.h.p.

Costo di ricerca. Analizzando il cammino all'indietro (dal basso verso l'alto): a ogni nodo, con prob. $\frac{1}{2}$ si è saliti (l'elemento era promosso) e con prob. $\frac{1}{2}$ si è andati a sinistra. Il numero atteso di passi per salire di un livello è 2 (variabile geometrica), e i livelli sono $O(\log n)$: il cammino di ricerca ha lunghezza attesa $O(\log n)$. $\square$

Domanda d'esame — Skip List

Prove that the average height/depth of a skip list with n nodes is $O(\log n)$.

Soluzione svolta

Teorema 6.10: la probabilità che un elemento arrivi al livello ℓ è $2^{-\ell}$, il numero atteso di elementi sopra il livello $2 \log_2 n$ è $\leq n \cdot 2^{-2 \log_2 n} = 1/n$, quindi per union bound l'altezza è $O(\log n)$ w.h.p.; il cammino di ricerca, analizzato all'indietro, sale di un livello in 2 passi

attesi su $O(\log n)$ livelli, da cui $O(\log n)$ atteso.

7 Strutture dati randomizzate: Treap (e Skip List)

Argomento del corso (slides di Erickson) presente come capitolo a sé negli appunti. La Skip List è già trattata nel cap. 6; qui sviluppiamo il Treap, l'altra struttura ad albero randomizzata, con cui spesso si risolvono gli esercizi "progetta una struttura dati".

7.1 Treap: definizione

Richiami di teoria

Definizione 7.1 (Treap). Un *treap* è un albero binario in cui ogni nodo ha una *chiave di ricerca* e una *priorità*: le chiavi rispettano la proprietà di *albero binario di ricerca* (BST, in-order ordinato), mentre le priorità rispettano la proprietà di *min-heap* (ogni nodo ha priorità minore dei figli). È quindi *simultaneamente* un BST sulle chiavi e un heap sulle priorità.

Idea della randomizzazione: le *priorità* sono assegnate *a caso* e indipendenti dalle chiavi. Con chiavi e priorità distinte, la struttura del treap è *univocamente determinata*: il nodo con *priorità minima è la radice (heap)*, le chiavi *minori vanno a sinistra* e le maggiori a destra (BST), e i sottoalberi sono treap per induzione. Equivalentemente: il treap è l'albero che si otterrebbe inserendo i nodi in un BST ordinario *in ordine di priorità crescente*. Poiché l'ordine delle priorità è una permutazione casuale, il treap si comporta come un *BST costruito da inserimenti in ordine casuale*, che ha altezza attesa $O(\log n)$.

Richiami di teoria — altezza attesa

Teorema 7.2 (Profondità attesa). In un treap con n nodi e priorità casuali distinte, la *profondità attesa di ogni nodo è $O(\log n)$* ; quindi *search/insert/delete costano $O(\log n)$ attesi*.

Idea. Il nodo x è antenato del nodo y (con ranghi $i < j$ nelle chiavi) se e solo se x ha la *priorità minima fra tutti i nodi di chiave compresa fra x e y inclusi*; con priorità casuali ciò accade con probabilità $\frac{1}{|i-j|+1}$. La *profondità attesa di un nodo è quindi $\sum_k \frac{1}{|i-k|+1} = O(H_n) = O(\log n)$* (serie armonica), esattamente come per un BST costruito in ordine casuale. $\square$

7.2 Operazioni (con rotazioni)

Richiami di teoria

La *rotazione* è l'operazione locale $O(1)$ che cambia la profondità di un nodo di ± 1 *preservando la proprietà di BST* (una rotazione destra in x e una sinistra in y sono inverse).

- **Search:** come in un BST ordinario, $O(\log n)$ atteso.
- **Insert(k):** assegna a k una priorità casuale, inseriscilo come foglia col solito inserimento BST (le chiavi restano valide); poi, finché k ha priorità minore del padre, *ruota k verso l'alto* per ripristinare l'heap. Costo $\propto$ profondità = $O(\log n)$ atteso.
- **Delete(k):** trova k ; finché k non è foglia, *ruota verso l'alto il figlio con priorità minore* (spinge k in basso mantenendo l'heap), poi staccalo da foglia. $O(\log n)$ atteso.
- **Split/Merge:** $\text{split}(T, \pi)$ separa T in $T_<$ (chiavi $< \pi$) e $T_>$ (chiavi $> \pi$) inserendo una chiave-sentinella π con priorità $-\infty$ alla radice; *merge* è l'inverso. Entrambe $O(\log n)$.

attese.

Nota: i bound del Treap sono identici (incluse le costanti) a quelli della Skip List — sono due implementazioni intercambiabili di un dizionario ordinato randomizzato.

Domanda d'esame — Treap / struttura randomizzata

Definisci un treap, descrivi insert/delete tramite rotazioni e dimostra che l'altezza attesa è $O(\log n)$. (Variante “progetta una struttura dati”: usa un treap come priority search tree per una 3-sided range query.)

Soluzione svolta

Definizione: 7.1 (BST sulle chiavi + min-heap su priorità casuali). **Insert:** priorità casuale + inserimento BST come foglia + rotazioni verso l'alto finché viola l'heap. **Delete:** rotazioni verso il basso (promuovendo il figlio a priorità minore) finché diventa foglia, poi rimozione. **Altezza $O(\log n)$:** Teorema 7.2, profondità attesa $\sum_k \frac{1}{|i-k|+1} = O(\log n)$ perché x è antenato di y con probabilità $\frac{1}{(\text{dist. rank})+1}$. **3-sided range query** $[x_1, x_2] \times (-\infty, y]$: tieni un treap con BST sulle ascisse x e heap (priorità =) sulle ordinate y (priority search tree). Per rispondere: scendi sui cammini di ricerca di x_1 e x_2 e riporta i punti con y entro la soglia sfruttando il vincolo di heap (ci si ferma appena y supera la soglia). Costo $O(\log n + k)$, output-sensitive con k punti riportati.

8 Prefix Search: trie, front coding, interpolation search

Richiami di teoria — il problema

Dato un dizionario $\mathcal{D}$ di n stringhe (lunghezza totale N , alfabeto σ), la *prefix search* per P trova tutte le stringhe di $\mathcal{D}$ che hanno P come prefisso. Riducibile alla ricerca di P come prefisso fra le stringhe ordinate. Obiettivo: poche I/O su disco, sfruttando blocchi e compressione.

8.1 Memorizzazione delle stringhe e Front Coding

Richiami di teoria — array di puntatori, due livelli, front coding

Array di puntatori a stringhe. Ricerca $O(p \log n)$ tempo, $O(\frac{p}{B} \log n)$ I/O; ma recuperare n_{occ} stringhe costa $\Omega(n_{occ})$ I/O perché sono sparse (un I/O per stringa).

Two-level storage. Si memorizzano le stringhe *contigue* in ordine lessicografico, a blocchi di taglia B ; un array A punta alla *prima* stringa di ogni blocco (le “stringhe campione”, $n_B \leq N/B$). Fase 1: binary search sui campioni ($O(\frac{p}{B} \log \frac{N}{B})$ I/O); Fase 2: scansione del blocco ($O(N_{occ}/B)$ I/O).

Front Coding (FC). Ogni stringa s_i è codificata come $(\ell_i, \hat{s}_i)$: ℓ_i = lunghezza del prefisso condiviso con s_{i-1} , $\hat{s}_i$ = suffisso residuo. La prima stringa di ogni blocco è memorizzata intera (per consentire l'accesso casuale al blocco). Riduce molto lo spazio quando stringhe vicine condividono lunghi prefissi.

Teorema 8.1 (Prefix search con FC a blocchi). *Con front coding a blocchi FC_B : ricerca $O(\frac{p}{B} \log \frac{FC_B(\mathcal{D})}{B})$ I/O; recupero $O(\frac{FC_B(\mathcal{D}_{occ})}{B})$ I/O.*

8.2 Trie compatta vs Patricia trie

Richiami di teoria

La *trie* (non compatta) su $\mathcal{D}$ ha $\leq N$ nodi interni e spazio $O(N\sigma)$ (o $O(N)$ con hashing nei nodi). La *trie compatta* (Patricia/radix) *comprime ogni cammino unario* (catena di nodi con un solo figlio) *in un unico arco etichettato da una sottostringa* (trippla $\langle s, i, j \rangle$ che punta dentro al testo): *scende a $O(n)$ nodi e $O(n)$ spazio.*

Teorema 8.2 (Trie compatta). *La trie compatta risponde alla prefix search in $O(p + n_{occ})$ tempo ($O(p + n_{occ}/B)$ I/O), con $O(n)$ nodi e spazio.*

Patricia trie (blind trie). Si spinge la compressione oltre: *ogni arco conserva solo il primo carattere*, e ogni nodo memorizza la *lunghezza* (branching position) della stringa fino a quel punto. *La ricerca è blind*: si discende guidati dal solo primo carattere di ogni arco, *senza* confrontare le sottostringhe; *alla fine si recupera una sola stringa candidata* (una foglia) e la si confronta interamente con P per validare.

Teorema 8.3 (Blind search). *La ricerca nel Patricia trie accede a una sola stringa del dizionario (1 I/O), indipendentemente da $|P|$, una volta percorso l'albero (che sta in memoria interna).*

Differenza fondamentale (da ricordare).

	Trie compatta	Patricia trie
Etichetta arco	sottostringa intera $\langle s, i, j \rangle$	solo primo carattere + lunghezza
Confronti in discesa	confronta le sottostringhe ($O(p)$ caratteri)	solo 1 carattere per arco (blind)
Accessi a disco alle stringhe	possibili più accessi	1 solo accesso (validazione finale)
Spazio	$O(n)$ nodi (+ stringhe)	$O(n)$, etichette ridotte a 1 carattere

Come si sfrutta nel two-level index. La Patricia trie **indicizza in memoria interna solo le stringhe campione $\mathcal{D}_B$** (le prime di ogni blocco); la blind search individua il blocco giusto con $O(p)$ lavoro in CPU e $O(1)$ I/O, poi si scandisce/decomprime (front coding) il blocco su disco con $O(N_{occ}/B)$ I/O. È proprio la blindness (1 solo accesso alla stringa) a rendere la Fase 1 a costo $O(1)$ I/O, cosa impossibile con la trie compatta che dovrebbe confrontare sottostringhe potenzialmente su disco.

Domanda d'esame — Compacted vs Patricia search

; Patricia

Describe the difference between a Compacted Trie and a Patricia Trie (space and query time as functions of n , total length N , pattern length P) and how this difference is exploited in the two-level string index. ([230724]: describe how to use a Patricia trie to find the lexicographic position of P and state the time/IO complexity.)

Soluzione svolta

Differenza: la trie compatta etichetta gli archi con sottostringhe intere e durante la discesa confronta $O(p)$ caratteri, potendo accedere più volte alle stringhe; la Patricia trie tiene sugli archi solo il primo carattere + la lunghezza, fa *blind search* (1 carattere per arco) e accede a *una sola* stringa per validare (Teoremi 8.2, 8.3). Entrambe usano $O(n)$ nodi/spazio (le stringhe a parte). **Sfruttamento nel two-level index:** la Patricia indicizza le stringhe campione in RAM; la blind search dà il blocco in $O(p)$ tempo e $O(1)$ I/O, poi la Fase 2 scandisce/decomprime (front coding) il blocco su disco in $O(N_{occ}/B)$ I/O. **Ricerca lessicografica [230724]:** blind descent guidata dal primo carattere fino a una foglia candidata, 1 accesso alla stringa per confronto completo, da cui si ricava la posizione lessicografica: $O(p)$ tempo, $O(1)$ I/O alla stringa.

8.3 Interpolation Search**Richiami di teoria**

Per m interi ordinati $x_1 < \dots < x_m$ si stima la **posizione del target y assumendo distribuzione (quasi) uniforme**: con **ampiezza media di bin $b = \frac{x_m - x_1 + 1}{m}$** , il bin candidato è $j = \lfloor \frac{y - x_1}{b} \rfloor + 1$, poi binary search dentro.

Teorema 8.4. *Interpolation search costa $O(\log \min(\Delta, m))$ nel caso peggiore, con $\Delta = \frac{\max_i(x_i - x_{i-1})}{\min_i(x_i - x_{i-1})}$ (rapporto fra gap massimo e minimo). Per interi uniformi in $[0, U)$ il costo è $O(\log \log m)$ con alta probabilità.*

Applicata alle stringhe campione di un two-level index, la Fase 1 scende a $O(\frac{p}{B} \log \log \frac{N}{B})$ I/O.

Domanda d'esame — Interpolation search

; storage di stringhe

Describe the interpolation-search data structure over a set and how a search is performed. ([240527]: given n strings of total length N , discuss ≥ 3 storage solutions, their space and the time/IO of $\text{ACCESS}(i)$.)

Soluzione svolta

Interpolation search: si parte dalla stima $j = \lfloor (y - x_1)/b \rfloor + 1$ del bin con b ampiezza media, poi binary search locale; costo $O(\log \log m)$ atteso su dati uniformi, $O(\log \min(\Delta, m))$ worst case (Teorema 8.4). **Tre soluzioni di storage [240527]:** (1) *Array di puntatori + stringhe sparse:* spazio $N + (1 + w)n$ byte, $\text{ACCESS}(i) = O(1)$ puntatore ma 1 I/O sparso per stringa; (2) *Stringhe contigue in ordine + array offset (two-level):* spazio $N + n \cdot w$, accesso sequenziale ai blocchi $O(\frac{|s_i|}{B})$ I/O, recupero contiguo efficiente; (3) *Front coding a blocchi:* spazio $FC_B(\mathcal{D}) + \text{offset}$, ma $\text{ACCESS}(i)$ richiede di decomprimere dal capo del blocco ($O(B)$ stringhe), 1 I/O per blocco — ottimo spazio, leggero costo in tempo. Il trade-off è spazio (FC) contro tempo d'accesso casuale (array di puntatori).

9 Substring Search: Suffix Array, LCP, Suffix Tree

Richiami di teoria — notazione

$T[1, n]$ su alfabeto Σ ($|\Sigma| = \sigma$), con $T[n] = \$$ sentinella (il carattere più piccolo). $\text{suffix}_i = T[i, n]$ è l' i -esimo suffisso. Cercare P come *sottostringa* di T equivale a cercarlo come *prefisso* di qualche suffisso di T .

9.1 Definizioni: Suffix Array e LCP array

Richiami di teoria

Definizione 9.1 (Suffix Array). $SA[1, n]$ è la permutazione delle posizioni $\{1, \dots, n\}$ che ordina i suffissi lessicograficamente:

$$\text{suffix}_{SA[1]} < \text{suffix}_{SA[2]} < \dots < \text{suffix}_{SA[n]}.$$

Occupa n interi in $[1, n]$, cioè $\Theta(n \log n)$ bit. Es. $T = \text{mississippi}\$$: $SA = [12, 11, 8, 5, 2, 1, 10, 9, 7, 4, 6, 3]$.

Definizione 9.2 (LCP array). $\text{lcp}[i] = \text{lcp}(\text{suffix}_{SA[i]}, \text{suffix}_{SA[i+1]})$ è la lunghezza del più lungo prefisso comune fra due suffissi *adiacenti* nell'ordine di SA . Ogni valore è in $[0, n)$.

Domanda d'esame — definizioni SA + LCP

Define formally what is the suffix array SA of a text $T[1, n]$ and define formally its corresponding LCP array.

Soluzione svolta

Definizioni 9.1 e 9.2: SA è la permutazione che ordina lessicograficamente i suffissi $\text{suffix}_{SA[1]} < \dots < \text{suffix}_{SA[n]}$; $\text{lcp}[i]$ è l'LCP fra i due suffissi consecutivi $\text{suffix}_{SA[i]}$ e $\text{suffix}_{SA[i+1]}$. (Conviene aggiungere l'esempio $\text{mississippi}\$$ per fissare le idee.)

9.2 Ricerca del pattern e COUNT

Richiami di teoria — binary search su SA

Le posizioni dei suffissi con prefisso P formano un *intervallo contiguo* $SA[i, j]$ (perché SA è ordinato): si trovano i due estremi con due binary search.

```
L ← 1; R ← n
while L != R:
    M ← floor((L+R)/2)
    if strncmp(P, suffix[SA[M]], p) > 0: L ← M+1
    else: R ← M
if strncmp(P, suffix[SA[L]], p) = 0: return L else return -1
```

Lemma 9.3 (Conteggio base). *Contare le occorrenze di $P[1, p]$ costa $O(p \log n)$ tempo e $O(\log n)$ accessi; recuperare le occ posizioni aggiunge $O(\text{occ})$. Spazio $\Theta(n(\log n + \log \sigma))$ bit.*

Proof. La binary search fa $O(\log n)$ iterazioni; ogni confronto fra P e un suffisso è $O(p)$ caratteri: $O(p \log n)$. L'intervallo trovato $SA[i, j]$ dà $occ = j - i + 1$ occorrenze, alle posizioni $SA[i], \dots, SA[j]$. $\square$

Richiami di teoria — ricerca ottimizzata con LCP

Mantenendo array lcp e i valori $Llcp[M] = lcp(suff_{SA[L]}, suff_{SA[M]})$, $Rlcp[M] = lcp(suff_{SA[M]}, suff_{SA[R]})$ (costruibili in $O(n)$ con $lcp[L, R] = \min\{lcp[L, M], lcp[M, R]\}$), e i valori correnti $\ell = lcp(P, suff_{SA[L]})$, $r = lcp(P, suff_{SA[R]})$, si evitano i confronti già noti.

Lemma 9.4 (Ricerca LCP-ottimizzata). *Contare $P[1, p]$ costa $O(p + \log n)$ nel caso peggiore ($+O(occ)$ per il recupero); ottimo nel modello a confronti.*

Idea. Supponiamo $\ell \geq r$. Confrontando ℓ con $Llcp[M]$: (i) $\ell < Llcp[M] \Rightarrow P > suff_{SA[M]}$, si va a destra *senza* confronti di caratteri; (ii) $\ell > Llcp[M] \Rightarrow P < suff_{SA[M]}$, si va a sinistra *senza* confronti; (iii) $\ell = Llcp[M] \Rightarrow$ si riprende il confronto dalla posizione $\ell + 1$. Ogni passo o dimezza l'intervallo *senza* leggere caratteri ($O(\log n)$ volte), oppure avanza di almeno un carattere di P ($\leq p$ volte). Totale $O(p + \log n)$. $\square$

Domanda d'esame — ricerca/COUNT su SA

Show how to COUNT all occurrences of $P[1, p]$ in T using an in-memory Suffix Array built off-line; show and prove the time complexity. What is the I/O-cost of RETRIEVING the positions of all occurrences when the SA is stored on disk?

Soluzione svolta

COUNT (in memoria): due binary search su SA per gli estremi dell'intervallo $SA[i, j]$; $occ = j - i + 1$. Complessità $O(p \log n)$ con il confronto ingenuo (Lemma 9.3), oppure $O(p + \log n)$ con gli array $Llcp/Rlcp$ (Lemma 9.4). La prova: $O(\log n)$ iterazioni $\times O(p)$ caratteri per confronto, ridotto a $O(p + \log n)$ ammortizzando i caratteri già confrontati.

RETRIEVAL su disco: dopo aver individuato l'intervallo $SA[i, j]$, le occ posizioni $SA[i..j]$ sono *contigue* nell'array su disco, quindi si leggono con scansione sequenziale: $O(\frac{occ}{B})$ I/O (più $O(\log n)$ I/O per le binary search, ciascuna che salta su pagine diverse: $O(\frac{p}{B} \log n)$ se anche T è su disco). Il punto chiave: il *count* è logaritmico, ma il *retrieval* è lineare in occ ed è efficiente in I/O solo perché le posizioni stanno in un tratto contiguo di SA .

9.3 Costruzione del Suffix Array

Richiami di teoria — approccio comparison-based

Teorema 9.5 (Costruzione via qsort). *Ordinare i suffissi con un **qsort** che confronta stringhe ha costo worst-case $O(n^2 \log n)$ tempo (confronti su stringhe lunghe fino a $\Theta(n)$) ed è I/O-inefficiente ($O(\frac{n}{B} n \log n)$ I/O) per due motivi: (a) i confronti coinvolgono stringhe di lunghezza variabile fino a $\Theta(n)$; (b) l'ordinamento permuta i puntatori, distruggendo la località.*

Se il massimo LCP fra suffissi è limitato da una costante c , ogni confronto legge $O(c)$ caratteri e il costo scende a $O(n \log n)$ confronti di caratteri.

Richiami di teoria — algoritmo Skew / DC3 (lineare)

DC3 (Difference Cover mod 3, Kärkkäinen–Sanders): riduce la costruzione del SA all'ordinamento di *triple di interi piccoli*, ottenendo tempo $O(n)$ (RAM) e $O(\text{sort}(n))$ I/O.

1. **Passo 1** — $SA^{2,0}$. Considera le posizioni $P_{2,0} = \{i : i \bmod 3 \in \{2, 0\}\}$. Forma una stringa $T^{2,0}$ di lunghezza $\frac{2}{3}n$ codificando i suffissi di quelle posizioni con triple di caratteri consecutivi; *ricorsivamente* calcola il suo SA e ricava $SA^{2,0}$.
2. **Passo 2** — SA^1 . Per le posizioni $P_1 = \{i : i \bmod 3 = 1\}$, rappresenta suff_i con la coppia $\langle T[i], \text{rank}(i+1) \rangle$ (dove $i+1 \in P_{2,0}$ ha già un rank dal Passo 1) e ordina le $O(n)$ coppie con RadixSort.
3. **Passo 3** — **Merge lineare**. Fondi $SA^{2,0}$ e SA^1 confrontando un suffisso di P_1 con uno di $P_{2,0}$ in $O(1)$ grazie ai rank precalcolati: per confronto con posizione $j \bmod 3 = 0$ usa la tripla $\langle T[i], T[i+1], \text{rank}(i+2) \rangle$; per $j \bmod 3 = 2$ usa la coppia $\langle T[i], \text{rank}(i+1) \rangle$.

Ricorrenza $T(n) = T(\frac{2}{3}n) + O(n) = O(n)$ (il " $\frac{2}{3} : \frac{1}{3}$ " è lo *skew*).

Richiami di teoria — costruzione dell'LCP: Kasai

Teorema 9.6 (Kasai). *Dato T e SA , l'array `lcp` si calcola in $O(n)$ tempo e spazio.*

Proof. Si scandiscono i suffissi nell'ordine *del testo* ($i = 1, \dots, n$), tenendo $h = \text{LCP}$ corrente. Fatto chiave: se i suffissi $\text{suff}_{p-1}, \text{suff}_p$ (in posizioni adiacenti di SA) hanno $\text{LCP } h > 0$, allora $\text{suff}_{p-1+1}, \text{suff}_{p+1}$ hanno $\text{LCP} \geq h - 1$. Quindi passando dal suffisso i al $i+1$ si può *decrementare* h al più di 1 e poi estendere con un while. Su n iterazioni, h viene decrementato $\leq n$ volte e incrementato in totale $\leq 2n$ volte (perché $0 \leq h \leq n$): il numero di confronti è $O(n)$.

```

h ← 0
for i = 1..n:
    q ← SA-1[i]      // rango del suffisso che inizia in i
    if q > 1:
        k ← SA[q-1]
        if h > 0: h ← h-1
        while T[k+h] = T[i+h]: h++
        lcp[q-1] ← h

```

Nota I/O: Kasai accede a `lcp` in ordine sparso (via SA^{-1}), quindi è $O(n)$ I/O nel caso peggiore — lineare in tempo ma non I/O-efficiente. $\square$

Domanda d'esame — costruzione SA via qsort

Give the pseudocode to construct the suffix array via qsort and state/comment the worst-case I/O complexity; what is the complexity when the maximum LCP among suffixes is bounded by a constant c ?

Soluzione svolta

qsort: si ordina l'array di indici $1..n$ usando come comparatore `strncmp(suff[a], suff[b])`. Worst case: confronti su stringhe lunghe $\Theta(n)$, quindi $O(n^2 \log n)$ tempo e I/O-inefficiente ($O(\frac{n}{B} \log n)$) per stringhe lunghe + permutazione dei puntatori che rovina la località (Teorema 9.5). **Con $\max \text{LCP} \leq c$:** ogni confronto legge $O(c) = O(1)$ caratteri prima di decidere, quindi il costo scende a $O(n \log n)$ confronti — come un ordinamento ordinario. (Per il caso generale lineare si usa DC3/Skew, $O(n)$ tempo, $O(\text{sort}(n))$ I/O.)

9.4 Suffix Tree

Richiami di teoria

Il *suffix tree* è la **trie compatta su tutti i suffissi di T** : ha esattamente n foglie (una per suffisso), ogni nodo interno (tranne la radice) ha ≥ 2 figli, ogni arco è etichettato da una sottostringa non vuota (coppia (i, j) che punta dentro T). Concatenando le etichette radice→foglia si ottiene un suffisso. Ha $\leq n - 1$ nodi interni e spazio $O(n)$.

- Una DFS in-order del suffix tree restituisce il SA ; viceversa, da $SA + \text{lcp}$ il suffix tree si costruisce in $O(n)$ (bottom-up con uno stack).
- **Applicazioni:** **pattern search** $O(p + \text{occ})$; ricerca della sottostringa più lunga ripetuta $\geq L$ volte; substring più lungo comune a più testi; ecc.
- In pratica costa 10–20 byte/carattere contro 4–8 del SA : il SA è preferito quando lo spazio conta.

Domanda d'esame — Suffix Tree

Using a suffix tree, how do you compute the longest substring occurring at least L times in T , and what is its time complexity in terms of n ?

Soluzione svolta

Si annota ogni nodo interno con il numero di foglie del suo sottoalbero (= numero di occorrenze della stringa rappresentata dal cammino radice→nodo); ciò si fa con una sola DFS in $O(n)$. La sottostringa cercata corrisponde al nodo interno *più profondo* (massima string-depth, cioè lunghezza della stringa-cammino) il cui sottoalbero ha $\geq L$ foglie. Tempo totale $O(n)$ (costruzione del suffix tree in $O(n)$ + una DFS in $O(n)$).

10 Integer Coding: numero di bit emessi

Questo è l'argomento più "a conti" del corso: la domanda tipica è "quanti bit emette il codice X sull'intero/sequenza Y ". Qui trovi, per ogni codice, la **formula esatta della lunghezza** e un **esempio svolto bit-per-bit**, più un esercizio completo su una sequenza.

10.1 Formule di lunghezza e codifiche bit-per-bit

Richiami di teoria — lunghezza ideale

La lunghezza **ideale** (Shannon) per un intero **con probabilità** $\mathcal{P}[x]$ è $L(x) = \log_2 \frac{1}{\mathcal{P}[x]}$ bit. Ogni codice è "ottimo" per la distribuzione che rende $L(x)$ uguale alla sua lunghezza effettiva. Notazione: $B(x)$ = rappresentazione binaria di x , $|B(x)| = \lceil \log_2(x+1) \rceil$.

Richiami di teoria — Unario e lunghezza fissa

Unario $U(x)$ ($x \geq 1$): $x - 1$ zeri seguiti da un 1 delimitatore.

$$|U(x)| = x \text{ bit.} \quad U(1) = 1, \quad U(4) = 0001, \quad U(6) = 000001.$$

Ottimo per $\mathcal{P}[x] = 2^{-x}$. **Lunghezza fissa**: con massimo m , $\lceil \log_2(m+1) \rceil$ bit per intero; ottimo per la distribuzione uniforme.

Richiami di teoria — Elias γ e δ (con esempi)

γ -codice: $|B(x)| - 1$ zeri seguiti da $B(x)$ (l'1 di testa di $B(x)$ fa da delimitatore). *Lunghezza*

$$|\gamma(x)| = 2|B(x)| - 1 = 2\lceil \log_2(x+1) \rceil - 1.$$

Esempio $x = 9$: $B(9) = 1001$ ($|B| = 4$), quindi $\gamma(9) = \underbrace{000}_{3 \text{ zeri}} \underbrace{1001}_{B(9)} = 0001001$, 7 bit. Ottimo per $\mathcal{P}[x] \approx 1/x^2$.

δ -codice: $\gamma(|B(x)|)$ seguito da $B(x)$ *intero* (le due parti non condividono bit). *Lunghezza*

$$|\delta(x)| = |\gamma(|B(x)|)| + |B(x)| \approx \log_2 x + 2 \log_2 \log_2 x + 1.$$

Esempio $x = 14$: $B(14) = 1110$, $\ell = |B(14)| = 4$, $\ell' = |B(4)| = 3$. Allora $\gamma(4) = \underbrace{00}_{\ell'-1} \underbrace{100}_{B(4)} = 00100$, e

$$\delta(14) = \underbrace{00}_{\gamma(4)=00100} \underbrace{100}_{B(4)} \underbrace{1110}_{B(14)} = 001001110, \quad 9 \text{ bit.}$$

Attenzione: nel corso δ mantiene il **$B(x)$ completo** (non si toglie l'1 di testa): quindi $\delta(14) = 9$ bit (non 8). Ottimo per $\mathcal{P}[x] \approx \frac{1}{x(\log x)^2}$, preferibile a γ per interi grandi.

Richiami di teoria — Rice, Variable-Byte, (s, c) -dense

Rice $R_k(x)$ ($x > 0$): quoziente $q = \lfloor (x-1)/2^k \rfloor$ in unario ($q+1$ bit) + resto $r = (x-1) - 2^k q$ in binario su k bit.

$$|R_k(x)| = q + k + 1.$$

Esempio $x = 83$, $k = 4$: $q = \lfloor 82/16 \rfloor = 5$, $r = 82 - 16 \cdot 5 = 2$, quindi $R_4(83) = U(6) \cdot B_4(2) = 0000010010$, 10 bit. Ottimo per la geometrica con $2^k \approx 0.69 \cdot \text{media}$.

Variable-Byte: si fa padding di $B(x)$ a multiplo di 7 bit, si spezza in blocchi da 7, e a ogni blocco si antepone 1 bit di flag (0=ultimo byte/stopper, 1=continuer). $|VB(x)| = 8 \lceil |B(x)|/7 \rceil$ bit (minimo 8). **(s,c)-dense:** $s + c = 2^b$; un byte codifica i primi s interi, due byte $s \cdot c$ interi, k byte fino a $s \frac{c^k - 1}{c - 1}$ interi. Ogni “cifra” è su b bit: s valori sono *stopper* (chiudono il codeword), c sono *continuer*. Un codeword è una sequenza di continuer seguita da uno stopper. *Primi 7 codeword con $s = 1, c = 3$ (esame [250110]):* qui $b = 2$ ($s + c = 4 = 2^2$); diciamo cifra 0 = stopper e cifre $\{1, 2, 3\} = \text{continuer}$ (digit su 2 bit):

1: 00 2: 01 00 3: 10 00 4: 11 00
5: 01 01 00 6: 01 10 00 7: 01 11 00

(1 valore a 1 byte, $s \cdot c = 3$ valori a 2 byte, poi $s \cdot c^2 = 9$ a 3 byte, ...).

10.2 Codice interpolativo (esempio svolto)

Richiami di teoria

Per una sequenza *crescente* $S' = (s'_1 < \dots < s'_n)$ (ottenuta da somme prefisse): a ogni passo, dato l'intervallo $[low, hi]$ e gli indici $[l, r]$, si prende il **mediano $m = \lfloor (l + r)/2 \rfloor$** e si **codifica s'_m con $\lceil \log_2 B \rceil$ bit**, dove $B = hi - low - r + l + 1$ è il numero di valori possibili per s'_m ; il **valore emesso è $s'_m - (low + m - l)$** . Si ricorre su sinistra $[low, s'_m - 1]$ e destra $[s'_m + 1, hi]$. Se la sottosequenza è $(low, low + 1, \dots)$, **$B = 1$ e non si emette nulla**. Non è prefix-free; per cluster densi può battere Huffman.

Domanda d'esame — Interpolative coding

Compress the integers (2, 5, 7) (oppure 4, 6, 11 in $S = (1, 4, 5, 6, 9, 11, 15)$) via Interpolative Coding.

Soluzione svolta

Prendiamo $S' = (2, 5, 7)$ con low = valore minimo possibile e hi massimo (qui assumiamo l'universo dato dall'esercizio, es. $[1, 8]$). Indici $l = 1, r = 3, low = 1, hi = 8$.

- **Mediano** $m = 2, s'_2 = 5$. $B = hi - low - r + l + 1 = 8 - 1 - 3 + 1 + 1 = 6$, servono $\lceil \log_2 6 \rceil = 3$ bit; valore emesso = $s'_2 - (low + m - l) = 5 - (1 + 1) = 3 = 011$.
- **Sinistra** $[l=1, r=1]$, $low = 1, hi = s'_2 - 1 = 4$, elemento $s'_1 = 2$: $B = 4 - 1 - 1 + 1 + 1 = 4$, $\lceil \log_2 4 \rceil = 2$ bit; valore $2 - (1 + 0) = 1 = 01$.
- **Destra** $[l=3, r=3]$, $low = s'_2 + 1 = 6, hi = 8$, elemento $s'_3 = 7$: $B = 8 - 6 - 3 + 3 + 1 = 3$, $\lceil \log_2 3 \rceil = 2$ bit; valore $7 - (6 + 0) = 1 = 01$.

Output = 0110101, totale 7 bit. **Metodo da ricordare:** sempre mediano prima, $B = hi - low - r + l + 1$, emetti lo scarto dal lower bound, ricorri; i tratti contigui costano 0 bit.

10.3 Elias–Fano: spazio, esempio svolto e Access

Richiami di teoria

n interi monotoni in $[0, u)$, $\ell = \lfloor \log_2(u/n) \rfloor$. **Bit bassi**: gli ℓ LSB di ogni s'_i , espliciti in L ($n\ell$ bit). **Bit alti** $h_i = \lfloor s'_i/2^\ell \rfloor$: in H si scrivono, per ogni bucket $v = 0, 1, \dots, \frac{u}{2^\ell} - 1$, tanti 0 quanti gli s'_i con $h_i = v$, poi un 1 separatore. H ha n zeri e $u/2^\ell$ uni.

$$\text{bit totali} = \underbrace{n\ell}_L + \underbrace{n + \frac{u}{2^\ell}}_H \leq n \lfloor \log_2(u/n) \rfloor + 2n.$$

Teorema 10.1 (Spazio di Elias–Fano). *La codifica occupa al più $n \lfloor \log_2(u/n) \rfloor + 2n$ bit; ACCESS(j) in $O(1)$ con $o(n)$ bit ausiliari. (Il valore esatto è $n\ell + n + u/2^\ell$: nel conteggio d'esame usa questo.)*

Access(j): $s_j = (\text{Select}_0(H, j) - j) \cdot 2^\ell + L[j]$, cioè bit alti $h_j = \text{Select}_0(H, j) - j$ (posizione del j -esimo zero meno j) concatenati con gli ℓ bit bassi di $L[j]$.

Domanda d'esame — spazio EF

; costruzione su numeri

State and prove the space of Elias–Fano; build the encoding of $S' = (3, 7, 11, 14, 19, 25)$ with universe $u = 32$ computing ℓ , the low-bits array L and the high-bits sequence H .

Soluzione svolta

$n = 6$, $u = 32$, $\ell = \lfloor \log_2(32/6) \rfloor = \lfloor \log_2 5.33 \rfloor = 2$.

- **Bit bassi** ($\ell = 2$ LSB di ciascuno): $3 = 0011$, $7 = 0111$, $11 = 1011$, $14 = 1110$, $19 = 10011$, $25 = 11001$, quindi $L = (11, 11, 11, 10, 11, 01)$, $n\ell = 12$ bit.
- **Bit alti** $h_i = \lfloor s'_i/4 \rfloor$: $(0, 1, 2, 3, 4, 6)$. Bucket $v = 0..7$ (poiché $u/2^\ell = 32/4 = 8$): un 0 per ciascun elemento nel bucket, poi un 1:

$$H = \underbrace{01}_{v=0} \underbrace{01}_{v=1} \underbrace{01}_{v=2} \underbrace{01}_{v=3} \underbrace{01}_{v=4} \underbrace{1}_{v=5} \underbrace{01}_{v=6} \underbrace{1}_{v=7} = 01010101011011, \text{ 14 bit.}$$

(6 zeri + 8 uni = 14.)

- **Spazio totale** = $12 + 14 = \boxed{26}$ bit. (La stima $n \lfloor \log_2(u/n) \rfloor + 2n = 12 + 12 = 24$ è la forma asintotica; il conteggio esatto è $n\ell + n + u/2^\ell = 12 + 6 + 8 = 26$.)

Access(4): il 4-esimo zero in H è in posizione 7 (contando da 1: 0, 0, 0, 0...), $h_4 = \text{Select}_0(H, 4) - 4 = 7 - 4 = 3$, e $s_4 = 3 \cdot 4 + L[4]_{(10)} = 12 + 2 = 14$. ✓

10.4 Tabella di confronto: bit emessi

Richiami di teoria — quadro riassuntivo

Codice	Bit per intero x	Esempio
Unario $U(x)$	x	$U(6) = 000001$ (6)
Lunghezza fissa	$\lceil \log_2(m+1) \rceil$	—
$\gamma(x)$	$2\lceil \log_2(x+1) \rceil - 1$	$\gamma(9) = 0001001$ (7)
$\delta(x)$	$ \gamma(B(x)) + B(x) $	$\delta(14) = 001001110$ (9)
Rice $R_k(x)$	$\lfloor (x-1)/2^k \rfloor + k + 1$	$R_4(83) = 0000010010$ (10)
Variable-Byte	$8\lceil B(x) /7 \rceil$	$x < 128 \Rightarrow 8$ bit
Interpolativo	variabile (adattivo)	$(2, 5, 7) \Rightarrow 7$ bit
Elias-Fano	$n\ell + n + u/2^\ell$ totali	$(3, 7, 11, 14, 19, 25) \Rightarrow 26$

Procedura d'esame per “quanti bit”: (1) per i codici universali calcola $|B(x)|$ e applica la formula; (2) per Rice scegli/usa k , calcola q, r ; (3) per Elias-Fano calcola ℓ , conta $n\ell$ bit bassi e $n + u/2^\ell$ bit di H ; (4) per l'interpolativo svolgi la ricorsione mediano-first.

11 Statistical Coding: entropia, Huffman, Arithmetic

Richiami di teoria — entropia

Entropia di ordine 0 di una sorgente su alfabeto Σ :

$$\mathcal{H} = \sum_{\sigma \in \Sigma} \Pr[\sigma] \log_2 \frac{1}{\Pr[\sigma]}.$$

È massima ($\log_2 |\Sigma|$) quando i simboli sono equiprobabili, minima (0) quando uno ha probabilità 1. L'entropia di ordine k condiziona ogni simbolo al contesto dei k precedenti: $\mathcal{H}_k = \sum_{w \in \Sigma^k} \frac{|w|_S}{|S|} \sum_{\sigma} \Pr[\sigma | w] \log_2 \frac{1}{\Pr[\sigma | w]}$, con $\mathcal{H}_k \leq \mathcal{H}_{k-1} \leq \dots \leq \mathcal{H}_0$.

11.1 Huffman e sua ottimalità

Richiami di teoria — costruzione e ottimalità

Huffman costruisce un **albero binario prefix-free fondendo** ripetutamente i due simboli (o nodi) di probabilità minima; $|\Sigma|$ foglie, $|\Sigma| - 1$ nodi interni. Lunghezza media $L_C = \sum_{\sigma} L(\sigma) \Pr[\sigma]$.

Teorema 11.1 (Ottimalità di Huffman). *Per ogni codice prefix-free C' vale $L_{\text{Huff}} \leq L_{C'}$: Huffman è a lunghezza media minima.*

Per induzione su $n = |\Sigma|$. Due lemmi: (1) esiste un albero ottimo in cui i due simboli a probabilità minima x, y sono fratelli alla massima profondità; (2) “contraendo” x, y in un unico simbolo z con $\Pr[z] = \Pr[x] + \Pr[y]$ si ottiene un albero ridotto R con $L_T = L_R + \Pr[x] + \Pr[y]$. Base $n = 2$: 1 bit a testa, ottimo. Passo: sia H l'albero di Huffman e C uno ottimo, entrambi con x, y fratelli (lemma 1). Contraendo x, y in z si ottengono alberi R_H, R_C su $n - 1$ simboli, con $L_H = L_{R_H} + \Pr[x] + \Pr[y]$ e $L_C = L_{R_C} + \Pr[x] + \Pr[y]$. R_H è l'albero di Huffman su $n - 1$ simboli, quindi per ipotesi induttiva $L_{R_H} \leq L_{R_C}$, da cui $L_H \leq L_C$. Ma C era ottimo, quindi $L_H = L_C$: Huffman è ottimo. $\square$

Teorema 11.2 (Huffman vs entropia). $\mathcal{H} \leq L_{\text{Huff}} < \mathcal{H} + 1$.

Proof. Lower bound: teorema di Shannon (nessun codice prefix-free batte $\mathcal{H}$). Upper bound: scegliendo $\ell_{\sigma} = \lceil \log_2(1/\Pr[\sigma]) \rceil$ si ha $\sum 2^{-\ell_{\sigma}} \leq 1$ (Kraft), quindi esiste un codice prefix-free C con $L_C = \sum \Pr[\sigma] \ell_{\sigma} < \mathcal{H} + 1$; per ottimalità $L_{\text{Huff}} \leq L_C < \mathcal{H} + 1$. $\square$

Numero di bit emessi. Su un testo di n simboli, Huffman emette $n \cdot L_{\text{Huff}}$ bit (più il preambolo $\Theta(|\Sigma|)$), con $n\mathcal{H} \leq nL_{\text{Huff}} < n(\mathcal{H} + 1)$: cioè fra $n\mathcal{H}$ e $n\mathcal{H} + n$ bit. Lo spreco massimo è 1 bit/simbolo, grave solo se $\mathcal{H} \ll 1$.

Domanda d'esame — Huffman con entropia

Costruisci l'albero di Huffman per $\Sigma = \{a, b, c, d, e, f\}$ con $\mathcal{P} = (0.05, 0.10, 0.15, 0.30, 0.25, 0.15)$, calcola la lunghezza media del codeword e confrontala con l'entropia.

Soluzione svolta

Fusioni greedy (si fondono ogni volta i due minimi):

$$\begin{aligned} a(.05) + b(.10) &= ab(.15); & ab(.15) + c(.15) &= abc(.30); \\ f(.15) + e(.25) &= fe(.40); & d(.30) + abc(.30) &= dabc(.60); \\ fe(.40) + dabc(.60) &= radice(1.0). \end{aligned}$$

Profondità risultanti: $d = 2$, $e = 2$, $c = 3$, $f = 3$, $b = 3$, $a = 3$ (l'assegnazione esatta dei bit dipende dai pareggi, ma le *lunghezze* e la media sono invarianti). Lunghezza media:

$$L_{\text{Huff}} = 0.30(2) + 0.25(2) + 0.15(3) + 0.15(3) + 0.10(3) + 0.05(3) = \mathbf{2.45} \text{ bit/simbolo.}$$

Entropia:

$$\mathcal{H} = \sum_{\sigma} \mathcal{P}[\sigma] \log_2 \frac{1}{\mathcal{P}[\sigma]} \approx \mathbf{2.39} \text{ bit/simbolo.}$$

Verifica del Teorema 11.2: $2.39 \leq 2.45 < 3.39$. ✓ Lo “spreco” è $2.45 - 2.39 = 0.06$ bit/simbolo: piccolo perché la distribuzione è quasi uniforme ($\mathcal{H}$ non lontana da $\log_2 6 \approx 2.585$).

11.2 Canonical Huffman

Richiami di teoria

Perché: l'albero di Huffman esplicito è costoso da memorizzare e da navigare in decompressione (puntatori, cache miss). Il *Canonical Huffman* usa le *stesse lunghezze di codeword dell'Huffman ottimo, ma assegna le codeword in ordine canonico* (lunghezze non decrescenti, codeword consecutive in binario), eliminando l'albero.

Cosa si memorizza nel preambolo del file compresso:

- per ogni lunghezza ℓ : il numero $\text{num}[\ell]$ di simboli con quella lunghezza, e la prima codeword $\text{fc}[\ell]$ di quella lunghezza (ricavabile aritmeticamente: $\text{fc}[\ell] = (\text{fc}[\ell - 1] + \text{num}[\ell - 1]) \cdot 2$);
- l'elenco dei simboli ordinati per (lunghezza, ordine alfabetico): $\text{symb}[]$.

Spazio $\approx \max^2$ bit per $\text{fc} + (|\Sigma| + \max) \log_2(|\Sigma| + 1)$ per symb ; decodifica con ≤ 2 cache miss per simbolo.

Decodifica di un simbolo (pseudocodice):

```
len ← 1; code ← leggi 1 bit
while code < fc[len]:    // non ancora una codeword valida di lunghezza
    len?
    // in realtà: while code NON appartiene al range di lunghezza len
    code ← (code « 1) + leggi 1 bit; len++
offset ← code - fc[len]
return symb[ first[len] + offset ] // first[len]=indice del primo
simbolo di lung. len
```

Si accumulano bit aumentando len finché code cade nel range $[\text{fc}[\text{len}], \text{fc}[\text{len}] + \text{num}[\text{len}]]$; allora l' offset dà direttamente il simbolo in symb .

Domanda d'esame — Canonical Huffman

Explain why Canonical Huffman was introduced; specify the data structures kept in the compressed-file preamble; write the pseudocode to decompress one symbol.

Soluzione svolta

Perché: per evitare di memorizzare/navigare l'albero esplicito — il canonico usa le stesse lunghezze ottime ma codeword consecutive, decodificabili aritmeticamente con pochi cache miss. **Preambolo:** le lunghezze (via $\text{num}[\ell]$ e $\text{fc}[\ell]$, prima codeword di ogni lunghezza) e la lista dei simboli ordinati per lunghezza $\text{symb}[]$. **Decodifica:** pseudocodice del riquadro — si accumulano bit aumentando la lunghezza finché il valore corrente entra nel range $[\text{fc}[\ell], \text{fc}[\ell] + \text{num}[\ell])$, poi $\text{symb}[\text{first}[\ell] + (\text{code} - \text{fc}[\ell])]$.

Domanda d'esame — Canonical Huffman svolto

Symbols $\mathcal{P} = (a:0.36, b:0.18, c:0.15, d:0.07, e:0.07, f:0.17)$: construct the Canonical Huffman code step-by-step and decompress the first symbols of a bit sequence.

Soluzione svolta

Lunghezze di Huffman (fondendo i minimi: $d+e = 0.14$, $0.14+c = 0.29$, $b+f = 0.35$, $a+0.29 = 0.65$, $0.35+0.65 = 1$): si ottengono $L(a) = 1$, $L(b) = L(c) = L(f) = 3$, $L(d) = L(e) = 4$ (verifica: $\sum 2^{-L} = \frac{1}{2} + 3 \cdot \frac{1}{8} + 2 \cdot \frac{1}{16} = 1$ ✓; $L_{\text{med}} = 0.36(1) + (0.18+0.15+0.17)(3) + (0.07+0.07)(4) = 2.42$, contro $\mathcal{H} \approx 2.36$). **Array canonici** (lunghezze $\ell = 1, 3, 4$): $\text{num} = [\ell 1:1, \ell 3:3, \ell 4:2]$; $\text{symb} =$ simboli ordinati per (lunghezza, alfabeto) $= [a \mid b, c, f \mid d, e]$. **First codes fc:** $\text{fc}[1] = 0 = 0$; regola $\text{fc}[\ell] = (\text{fc}[\ell-1] + \text{num}[\ell-1]) \cdot 2$, con i livelli mancanti contati a 0: $\text{fc}[2] = (0+1) \cdot 2 = 2$, $\text{fc}[3] = (2+0) \cdot 2 = 4 = 100$, $\text{fc}[4] = (4+3) \cdot 2 = 14 = 1110$. Codeword consecutivi: $a=0$; $b=100$, $c=101$, $f=110$; $d=1110$, $e=1111$. **Decodifica** (accumula bit aumentando ℓ finché $\text{code} \in [\text{fc}[\ell], \text{fc}[\ell] + \text{num}[\ell])$): sul flusso 0111001...: primo bit 0, $\ell=1$, $0 \in [0, 1) \Rightarrow a$; poi 111, $\ell=3$, $7 \notin [4, 7)$? $7 \geq 4+3$ quindi sali $\ell=4$, leggi un bit $\rightarrow 1110 = 14 \in [14, 16) \Rightarrow d$; poi resta 0, $\ell=1$, $0 \in [0, 1) \Rightarrow a$. **Primi tre simboli:** a, d, a (consumati i bit 0 1110 0).

11.3 Arithmetic Coding

Richiami di teoria

Codifica l'intero testo come un singolo numero reale in $[0, 1)$. Partendo da $[lo, hi) = [0, 1)$, per ogni simbolo s_k si restringe l'intervallo proporzionalmente alla sua probabilità (cumulata $F(s) = \sum_{\sigma < s} \Pr[\sigma]$):

$$range = hi - lo, \quad hi \leftarrow lo + range \cdot (F(s_k) + \Pr[s_k]), \quad lo \leftarrow lo + range \cdot F(s_k).$$

Alla fine si emette un numero dentro $[lo, hi)$ con $\lceil \log_2 \frac{1}{hi-lo} \rceil + 1$ bit.

Teorema 11.3 (Bit emessi). *Arithmetic coding di un testo di n simboli con entropia $\mathcal{H}$ emette al più $n\mathcal{H} + 2$ bit.*

Idea. Dopo n simboli l'intervallo ha ampiezza $\prod_{k=1}^n \Pr[s_k]$, quindi servono $\lceil \log_2 \frac{1}{\prod_{k=1}^n \Pr[s_k]} \rceil + 1 = \lceil \sum_k \log_2 \frac{1}{\Pr[s_k]} \rceil + 1$ bit. Il termine sommatoria vale in media $n\mathcal{H}$ (per la legge dei grandi numeri / definizione di entropia empirica), e l'overhead di arrotondamento è ≤ 2 bit *in totale*: quindi $\leq n\mathcal{H} + 2$ bit. $\square$

Perché si parte “dal centro” e si tronca [210907]: non si usa l'estremo sinistro lo dell'intervallo finale perché potrebbe richiedere infiniti bit per essere rappresentato univocamente come prefisso; si sceglie un punto *interno* (il più corto binario dentro $[lo, hi)$) e lo si *tronca* alla precisione minima che lo mantiene dentro l'intervallo, garantendo decodifica univoca con $\lceil \log_2 \frac{1}{hi-lo} \rceil + 1$ bit.

Richiami di teoria — Huffman vs Arithmetic

- **Huffman:** $\mathcal{H} \leq L < \mathcal{H} + 1$ bit *per simbolo*; spreca fino a 1 bit/simbolo (grave quando $\mathcal{H} \ll 1$, es. alfabeti molto sbilanciati o binari).
- **Arithmetic:** $\leq n\mathcal{H} + 2$ bit *in totale*, cioè overhead $O(1/n)$ per simbolo: arriva praticamente all'entropia. Migliore con distribuzioni molto sbilanciate.
- **Tempo:** Huffman è più veloce (lookup di codeword, niente moltiplicazioni); Arithmetic richiede aritmetica a precisione finita (moltiplicazioni/rinormalizzazioni). Quindi: Huffman quando conta la velocità e $\mathcal{H}$ non è troppo piccola; Arithmetic quando conta il rapporto di compressione su sorgenti molto sbilanciate.

Domanda d'esame — Huffman vs Arithmetic; permutazione

State the number of output bits of Huffman vs Arithmetic given text length n and entropy $\mathcal{H}$; comment when each is preferable. ([231212], [241209]: prove the arithmetic-coding bit bound. [250623]: prove that two texts where one is a permutation of the other are compressed to the same number of bits by arithmetic coding.)

Soluzione svolta

Bit: Huffman $n\mathcal{H} \leq L_{\text{tot}} < n\mathcal{H} + n$ (fino a 1 bit/simbolo di spreco); Arithmetic $\leq n\mathcal{H} + 2$ (Teorema 11.3). **Quando:** Arithmetic vince su sorgenti molto sbilanciate ($\mathcal{H} \ll 1$), dove Huffman paga il bit intero per simbolo; Huffman vince in velocità. **Invarianza per permutazione [250623]:** con un modello statico basato sulle frequenze, l'ampiezza dell'intervallo finale è $\prod_{k=1}^n \Pr[s_k] = \prod_{\sigma} \Pr[\sigma]^{n_{\sigma}}$, che dipende solo dai *conteggi* n_{σ} dei simboli, non dal loro ordine. Due testi che sono uno la permutazione dell'altro hanno gli stessi conteggi, quindi lo stesso prodotto e lo stesso numero di bit emessi $\lceil \sum_k \log_2 \frac{1}{\Pr[s_k]} \rceil + 1$. (Nota: vale per il modello statico; con un modello adattivo l'ordine può cambiare leggermente il risultato.)

Domanda d'esame — Arithmetic coding svolto

Codifica la stringa $S = \mathbf{abbc}$ con $\mathcal{P}[a] = 0.5$, $\mathcal{P}[b] = 0.3$, $\mathcal{P}[c] = 0.2$. Mostra l'evoluzione di $[lo, hi)$ e i bit finali.

Soluzione svolta

Ordine $a < b < c$, cumulate $F(a) = 0$, $F(b) = 0.5$, $F(c) = 0.8$. Partendo da $[0, 1)$, ad ogni simbolo $hi \leftarrow lo + range \cdot (F + \mathcal{P})$, $lo \leftarrow lo + range \cdot F$:

simbolo	lo	hi
a	0	0.5
b	$0 + 0.5(0.5) = 0.25$	$0 + 0.5(0.8) = 0.40$
b	$0.25 + 0.15(0.5) = 0.325$	$0.25 + 0.15(0.8) = 0.370$
c	$0.325 + 0.045(0.8) = 0.361$	$0.325 + 0.045(1.0) = 0.370$

Intervallo finale $[0.361, 0.370)$, ampiezza 0.009. Bit emessi = $\lceil \log_2(1/0.009) \rceil + 1 = \lceil 6.80 \rceil + 1 = 8$ bit. (Confronto: 4 simboli con entropia $\mathcal{H} \approx 1.485$ darebbero $\approx n\mathcal{H} = 5.9$ bit più overhead; Huffman qui userebbe $a=1, b=2, c=2$ bit $\rightarrow 1+2+2+2 = 7$ bit, ma su sorgenti più sbilanciate l'aritmetico vince.)

12 Compressione basata su dizionario: LZ77, LZ78, LZW

Capitolo presente negli appunti (“Dictionary Based Compressors”). Raramente è domanda di sola teoria, ma l’esercizio di parsing esce e il confronto LZ77/LZ78 è uscito allo scritto 2024. Qui: metodo + esempio svolto per ciascun algoritmo.

Richiami di teoria — idea

A differenza della compressione statistica, i compressori a dizionario non stimano probabilità: costruiscono *dinamicamente* un dizionario di stringhe già viste e sostituiscono le occorrenze con un riferimento. Sono la base di gzip, 7zip, ZSTD, Brotli, LZ4, GIF. Sono compressori *universali*: al crescere dell’input il rapporto converge all’entropia della sorgente.

12.1 LZ77 (sliding window) e gzip

Richiami di teoria

Usa una *sliding window* W (parte già processata) e un *look-ahead buffer* B . **Parsing:** si cerca il più lungo prefisso α di B che occorre in $W \cdot B$; se a distanza d con lunghezza $|\alpha|$ e carattere seguente c , si emette la *trippla* $\langle d, |\alpha|, c \rangle$ e si avanza di $|\alpha| + 1$; se non c’è match, $\langle 0, 0, B[1] \rangle$. **Variante LZss:** emette *coppie* $\langle d, |\alpha| \rangle$ per i match e $\langle 0, c \rangle$ per i caratteri letterali (toglie la ridondanza della tripla). **gzip:** indicizza i 3-grammi di W con una hash table per trovare i match velocemente; codifica distanze e lunghezze con due alberi di Huffman; opzioni $-1 \dots -9$ controllano $|W|$ (100–900 KB). **Decompressione** velocissima: ogni $\langle d, \ell \rangle$ è una copia di ℓ byte a distanza d (for i in $0..L-1$: $S[s+i] = S[s-d+i]$).

Domanda d’esame — parsing LZ77/LZss

Esegui il parsing LZ77 (variante LZss) di una stringa; gestisci l’overlapping copy (match che si estende dal buffer nella finestra).

Soluzione svolta

Esempio $S = \text{aabbababbaabababbaa\$}$ (finestra illimitata):

$W \mid B$	output
aabbabab...	$\langle 0, 0, a \rangle$
a abbabab...	$\langle 1, 1, b \rangle$
aab bababab...	$\langle 1, 1, a \rangle$
aabba babab...	$\langle 2, 3, \dots \rangle$

Overlapping copy: quando $|\alpha| > d$ il match “entra” nel buffer non ancora scritto (es. copiare abab con $d = 2$): è corretto perché la decompressione copia *byte per byte* da sinistra a destra, quindi i byte appena scritti diventano sorgente per i successivi (la copia si “autoalimenta”, come per i run).

12.2 LZ78 (dizionario esplicito a trie)

Richiami di teoria

Dizionario esplicito $\mathcal{D}$ (inizialmente solo la stringa vuota, id 0). Ad ogni passo: (1) trova il più lungo prefisso α di B presente in $\mathcal{D}$; (2) sia c il carattere seguente, emette $\langle \text{id}(\alpha), c \rangle$; (3) aggiunge αc a $\mathcal{D}$ col prossimo id. Il dizionario è *prefix-complete* $\Rightarrow$ si implementa con un *trie non compattato* (ogni arco un simbolo), e la ricerca del prefisso è una discesa.

Domanda d'esame — parsing LZ78

Esegui il parsing LZ78 e costruisci il dizionario per $S = aabbababbbbaababaa$.

Soluzione svolta

frase	output	dizionario
a	$\langle 0, a \rangle$	1 : a
ab	$\langle 1, b \rangle$	2 : ab
b	$\langle 0, b \rangle$	3 : b
aba	$\langle 2, a \rangle$	4 : aba
bb	$\langle 3, b \rangle$	5 : bb
ba	$\langle 3, a \rangle$	6 : ba
abab	$\langle 4, b \rangle$	7 : abab
aa	$\langle 1, a \rangle$	8 : aa

Il trie ha un nodo per ogni frase (1–8); l'arco da $\text{id}(\alpha)$ etichettato c porta alla nuova frase αc .

12.3 LZW (solo id, dizionario inizializzato)

Richiami di teoria

Variante di LZ78 che emette *solo* l'id (niente carattere). Per renderlo possibile: il dizionario è inizializzato con tutti i singoli simboli ($\text{id} = 0..|\Sigma| - 1$); si emette $\text{id}(\alpha)$ e si aggiunge αc , ma la frase successiva *riparte da c* (parsing e aggiornamento *disallineati*). **Decodifica:** ricostruisce il dizionario incrementalmente; il caso speciale (la frase referencia una frase appena inserita e non ancora completa) si risolve notando che $f[1] = \alpha'[1]$ (il primo carattere coincide con quello della frase precedente). Base del formato GIF.

Domanda d'esame — LZW

Esegui la codifica LZW di $S = aabbabab$ ($a = 97, b = 98$).

Soluzione svolta

frase	output	dizionario
a	97	256 : aa
a	97	257 : ab
b	98	258 : bb
b	98	259 : ba
ab	257	260 : aba
aba	260	261 : abab

Stream compresso: `[97, 97, 98, 98, 257, 260]`. Si noti il disallineamento: dopo aver emesso 97 per la prima `a`, la frase successiva riparte dalla seconda `a`.

12.4 LZ77 vs LZ78/LZW (confronto)

Richiami di teoria		
	LZ77	LZ78/LZW
Dizionario	implicito (sliding window)	esplicito (trie)
Dimensione	limitata da $ W $	cresce con l'input
Output/frase	$\langle d, \ell, c \rangle$ o $\langle d, \ell \rangle$	$\langle \text{id}, c \rangle$ o solo <code>id</code>
Compressione	in genere migliore	leggermente peggiore
Vel. compress.	più lenta (ricerca in W)	più veloce (lookup trie)
Vel. decompress.	molto veloce	veloce
Strumenti	gzip, 7zip, ZSTD, Brotli	compress, GIF

Entrambe universali: il rapporto converge all'entropia della sorgente (ipotesi di ergodicità/stazionarietà). È la risposta alla domanda d'esame 2024 sul confronto.

13 Burrows–Wheeler Transform e pipeline bzip2

Nota. Negli esami la BWT compare quasi sempre come *esercizio* di calcolo (BWT+MTF+RLE0+Huffman su una stringa); non come domanda di sola teoria. La includiamo perché è centrale nel corso e le sue *proprietà* (LF-mapping, inversione, legame col Suffix Array) sono le fondamenta dell’FM-index del capitolo 14, dove invece le domande teoriche abbondano.

13.1 Trasformata diretta e inversa

Richiami di teoria — forward BWT

Si appende \$ (minimo) a S . Concettualmente si costruisce la matrice $(n+1) \times (n+1)$ $\mathcal{M}$ di tutte le rotazioni cicliche di $S\$$ e si ordinano le righe lessicograficamente, ottenendo $\mathcal{M}'$. La BWT è l’ultima colonna L di $\mathcal{M}'$ (più la posizione r della riga che termina in \$). $\mathcal{M}'$ si costruisce in $O(n)$ via Suffix Array (non serve materializzare $\mathcal{M}$, che è $\Theta(n^2)$). La trasformata tende a *raggruppare caratteri uguali* (local homogeneity), il che la rende molto comprimibile dopo MTF+RLE.

Richiami di teoria — inversa via LF-mapping

Siano F la prima colonna (i caratteri di $S\$$ ordinati) e L l’ultima (la BWT).

Teorema 13.1 (Proprietà 1). *Per ogni riga, $L[i]$ è il carattere che precede $F[i]$ nel testo originale (ciclicamente).*

Proof. La riga i è una rotazione ciclica $S[k_i] S[k_i+1] \dots$; il suo primo carattere è $F[i] = S[k_i]$ e l’ultimo è $L[i] = S[k_i - 1]$, ovvero il precedente. $\square$

Teorema 13.2 (Proprietà 2). *Per ogni carattere c , la k -esima occorrenza di c in L corrisponde alla k -esima occorrenza di c in F (stesso ordine relativo).*

Proof. Siano due righe che iniziano per c : $r(i) = c\alpha$ e $r(j) = c\beta$ con $i < j$, quindi $\alpha \prec \beta$. Ruotando a sinistra, in $\mathcal{M}'$ compaiono come αc e βc (che finiscono in c , cioè in L); poiché $\alpha \prec \beta$ l’ordine relativo è preservato. Dunque le occorrenze di c mantengono lo stesso ordine in F e in L . $\square$

LF-mapping: $LF[i] = j$ tale che il carattere $L[i]$ “è” lo stesso di $F[j]$; se $L[i]$ è la k -esima occorrenza di c in L , allora $F[LF[i]]$ è la k -esima occorrenza di c in F . Si calcola in $O(n)$:

```
// C[c] = posizione in F della prima occorrenza di c
conta n_c per ogni c (scansione di L)
C ← cumulative counts
for i = 1..n+1: LF[i] ← C[L[i]]; C[L[i]]++
```

Teorema 13.3 (Inversione). *S si ricostruisce da (L, r) in $O(n)$ tempo/spazio, con ≤ 1 cache miss per simbolo.*

Proof. Partendo dalla riga 1 (quella $S\$$) e applicando ripetutamente LF, si visitano le righe nell’ordine che ricostruisce S dall’ultimo carattere al primo: **k=1; for i=n downto 1: S[i]=L[k]; k=LF[k]**. Tre scansioni lineari per costruire LF + n iterazioni $O(1)$: totale $O(n)$. Con L, LF contigui, ≤ 1 cache miss per accesso. $\square$

Richiami di teoria — MTF, RLE, bzip2

Move-To-Front (MTF): lista dinamica inizializzata in ordine alfabetico; per ogni simbolo si emette la sua *posizione corrente* nella lista (da 0 o da 1 secondo convenzione) e lo si sposta in testa. Su L (omogenea a tratti) produce molti valori piccoli (spesso 0/1). **Run-Length Encoding (RLE):** ogni run massimale di un simbolo $\rightarrow$ coppia (simbolo, lunghezza). “RLE0” codifica solo i run di 0 (es. col codice di Wheeler). **Pipeline bzip2:** (RLE iniziale) $\rightarrow$ BWT $\rightarrow$ MTF $\rightarrow$ RLE $\rightarrow$ Huffman canonico (o aritmetico), su blocchi di ~ 900 KB compressi indipendentemente.

Teorema 13.4 (Legame con il Suffix Array). $F[i] = T[SA[i]]$ e $L[i] = T[SA[i] - 1]$ (e \$ quando $SA[i] = 1$): quindi la BWT si calcola in $O(n)$ da un Suffix Array costruito in tempo lineare.

Risultati teorici: la compressione basata su BWT è λ -ottima per ogni $k \geq 0$ (il rapporto converge a $H_k(S) + \varepsilon$), risultato più forte di LZ78; è invertibile in $O(n)$ con ≤ 1 cache miss per simbolo.

14 Strutture dati compresse: Rank/Select, FM-index, alberi succinti

14.1 Rank in $O(1)$ con $o(m)$ bit extra

Richiami di teoria — definizioni

Dato un array binario $B[1, m]$:

$$\text{Rank}_1(i) = \sum_{j=1}^i B[j], \quad \text{Rank}_0(i) = i - \text{Rank}_1(i), \quad \text{Select}_b(i) = \min\{j : \text{Rank}_b(j) = i\}.$$

Obiettivo: rispondere a Rank in $O(1)$ usando $o(m)$ bit in più oltre a B (che resta read-only). Si usa una struttura a tre livelli.

Richiami di teoria — struttura a 3 livelli per Rank

Siano $Z = (\log m)^2$ (big block) e $z = \frac{1}{2} \log m$ (small block).

- **Livello 1 — Big blocks.** Per ogni big block (taglia Z) si memorizza il Rank_1 assoluto fino al suo inizio. Sono m/Z valori da $\leq \log m$ bit: spazio $\frac{m}{Z} \log m = \frac{m}{(\log m)^2} \log m = \frac{m}{\log m} = o(m)$.
- **Livello 2 — Small blocks.** Dentro ogni big block, per ogni small block (taglia z) si memorizza il Rank_1 relativo all'inizio del big block. È $\leq Z = (\log m)^2$, quindi $O(\log \log m)$ bit ciascuno; sono m/z valori: spazio $\frac{m}{z} O(\log \log m) = O(\frac{m \log \log m}{\log m}) = o(m)$.
- **Livello 3 — Lookup table.** Tabella $R[b, o]$ = numero di 1 nei primi o bit della configurazione b , per ogni possibile small block (2^z configurazioni) e ogni offset $o \leq z$. Spazio $2^z \cdot z \cdot O(\log z) = 2^{\frac{1}{2} \log m} \cdot O(\dots) = \sqrt{m} \cdot O((\log m)(\log \log m)) = o(m)$.

Risposta a $\text{Rank}_1(x)$ in $O(1)$: si sommano i tre contributi

$$\text{Rank}_1(x) = \underbrace{r_{\text{big}}}_{\text{liv.1}} + \underbrace{r_{\text{small}}}_{\text{liv.2}} + \underbrace{R[B_{\text{small}}, o]}_{\text{liv.3}},$$

dove r_{big} è il rank assoluto del big block che contiene x , r_{small} il rank relativo dello small block, e l'ultimo termine il rank dentro lo small block letto dalla tabella (in pratica un'istruzione popcount). Ogni accesso è $O(1)$.

Teorema 14.1 (Rank succinto). *Esiste una struttura che risponde a Rank in $O(1)$ worst case usando $o(m)$ bit oltre a B .*

Perché $z = \frac{1}{2} \log m$ (non $\log m$)? Per avere $2^z = \sqrt{m} = o(m)$ configurazioni nella tabella di lookup: con $z = \log m$ si avrebbe $2^z = m$ e la tabella occuperebbe $\Theta(m)$ bit, rovinando la garanzia $o(m)$.

Domanda d'esame — Rank succinto

Describe the data structure supporting RANK over a binary array $B[1, m]$ in constant time, and prove its time and space complexity (in bits): $O(1)$ time, $o(m)$ extra bits.

Soluzione svolta

Struttura a tre livelli (Teorema 14.1): big block $Z = (\log m)^2$ con rank assoluto ($\frac{m}{\log m} = o(m)$ bit), small block $z = \frac{1}{2} \log m$ con rank relativo ($O(\frac{m \log \log m}{\log m}) = o(m)$ bit), lookup table di $2^z = \sqrt{m}$ configurazioni ($o(m)$ bit). $\text{Rank}_1(x) = r_{\text{big}} + r_{\text{small}} + R[B_{\text{small}}, o]$, somma di tre $O(1)$. **Spazio totale** = somma dei tre contributi $o(m)$, quindi $o(m)$ bit oltre a B ; **tempo** $O(1)$ worst case. Il punto cruciale nella prova dello spazio: $z = \frac{1}{2} \log m$ rende la tabella $2^z = \sqrt{m}$ (sotto-lineare), e $Z = (\log m)^2$ rende i rank relativi rappresentabili in $O(\log \log m)$ bit.

Richiami di teoria — Select

Select si divide per *numero di 1* invece che per posizione. Si raggruppano gli 1 a gruppi di $K = (\log m)^2$; per i gruppi “sparsi” (che coprono $> K^2$ bit) si memorizzano esplicitamente le K posizioni ($O(K \log m)$ bit per gruppo, $o(m)$ totale); per i gruppi “densi” si ricorre a una struttura analoga su gruppi più piccoli di $(\log \log m)^2$ uni.

Teorema 14.2 (Select succinto). Select in $O(1)$ tempo con $o(m)$ bit extra.

14.2 FM-index: Backward Search**Richiami di teoria — Count via backward search**

L’FM-index (Ferragina–Manzini) è un indice *compresso* di T basato su BWT + Rank. Usa $C[c] = 1 + \sum_{x < c} n_x$ (posizione in F del primo c) e $\text{Rank}(c, i) =$ numero di c in $L[1, i]$. Cerca P da destra a sinistra:

```
i ← p; c ← P[p]
first ← C[c]; last ← C[c+1] - 1
while first ≤ last and i > 1:
    c ← P[i-1]
    first ← C[c] + Rank(c, first - 1)
    last ← C[c] + Rank(c, last) - 1
    i ← i - 1
return (first, last)    // occ = last - first + 1
```

Invariante: all’iterazione i , $[first, last]$ è l’intervallo di righe di $\mathcal{M}'$ i cui prefissi coincidono con il suffisso $P[i, p]$. **Correttezza:** usa la Proprietà 2 della BWT (Teorema 13.2) e l’LF-mapping: $\text{Rank}(c, last) - \text{Rank}(c, first - 1)$ conta le occorrenze di c nell’intervallo corrente di L , che via LF mappano *esattamente* sulle righe il cui prefisso è $c \cdot P[i, p] = P[i - 1, p]$. Quindi a ogni passo l’intervallo si restringe correttamente al pattern esteso di un carattere a sinistra.

Teorema 14.3 (Complessità di Count). Count costa $O(p \cdot t_{\text{Rank}})$ tempo; spazio $nH_k(S) + o(n \log \sigma)$ bit per ogni $k = o(\log_\sigma n)$. Se $\sigma = O(\text{polylog } n)$ allora Rank = $O(1)$ e Count è ottimo $O(p)$.

Richiami di teoria — Locate ed Extract

Locate: si campionano le posizioni testuali ogni μ righe (sample); per una riga r generica si applica LF ripetutamente ($\text{LF}^t(r)$) finché si arriva a una riga campionata h , poi $\text{pos}(r) = \text{pos}(h) + t$. Costo $O(\mu \cdot \text{occ})$ tempo, $O(\frac{n}{\mu} \log n)$ bit extra; con $\mu = \log^{1+\varepsilon} n$: polylog per occorrenza, spazio sub-lineare. **Extract**(i, j): si ricostruisce $T[i, j]$ simulando la trasformata

inversa — si parte dalla riga campionata vicina a j e si applica LF (calcolato al volo via Rank) $j - i$ volte, emettendo un carattere per passo.

Domanda d'esame — FM-index / backward search

Show how to COUNT the occurrences of $P[1, p]$ in T using the FM-index backward search; state the invariant and prove correctness; give the time complexity.

Soluzione svolta

Pseudocodice di backward search del riquadro. **Invariante:** $[first, last]$ = righe di $\mathcal{M}'$ prefissate da $P[i, p]$. **Correttezza:** per estendere il pattern di un carattere $c = P[i - 1]$ a sinistra, le righe nuove sono quelle che iniziano per c e la cui LF-immagine cade nell'intervallo corrente; per la Proprietà 2 (ordine preservato fra F e L) e la definizione di LF, queste sono esattamente $C[c] + \text{Rank}(c, first - 1) .. C[c] + \text{Rank}(c, last) - 1$. **Complessità:** p iterazioni, ognuna due Rank: $O(p \cdot t_{\text{Rank}})$, cioè $O(p)$ quando σ è polylog (Teorema 14.3). $occ = last - first + 1$.

14.3 Alberi succinti (LOUDS / parentesi bilanciate)

Richiami di teoria — codifica e navigazione

Un albero di n nodi si codifica in $2n + O(1)$ bit. **LOUDS** (Level-Order Unary Degree Sequence): visita i nodi per livelli; per un nodo con d figli scrive d volte 1 seguito da uno 0 (grado in unario). Si naviga con Rank/Select sul bit-vector B , e si tengono le etichette in un array L separato (in ordine di visita). Per un albero binario si usa spesso una codifica con un bit per puntatore (1 = figlio presente, 0 = NULL).

Le *primitive* (tutte $O(1)$ con Rank/Select succinti):

- numero di 1 fino a una posizione = Rank_1 ; posizione del k -esimo 1 = Select_1 ;
- da un nodo ai suoi figli / al padre: formule con Rank/Select a seconda della codifica;
- un nodo è *foglia* se ha grado 0 (in LOUDS: due 0 consecutivi nel punto corrispondente; in codifica binaria: entrambi i bit dei figli a 0).

Domanda d'esame — pseudocodici su alberi succinti

Dato l'albero codificato succintamente in B (+ etichette in L), scrivi lo pseudocodice per: verificare l'esistenza di un cammino/arco; stampare le etichette delle foglie; contare i nodi interni; contare i nodi con una certa etichetta il cui "nonno" ha un'altra etichetta; scoprire se un nodo è foglia.

Soluzione svolta

Tutti questi esercizi seguono lo stesso schema: *navigare l'albero usando solo Rank/Select su B e leggere le etichette da L* . Esempi tipici (codifica binaria, $B[1, 2n+1]$ con due bit figli per nodo):

- **x è foglia?** I due bit dei figli di x sono entrambi 0 (nessun figlio): $O(1)$.
- **Conta nodi interni:** numero di nodi – numero di foglie; le foglie sono le posizioni con entrambi i bit-figlio a 0, contabili con Rank in $O(n)$ (una scansione) o $O(1)$ con struttura

ausiliaria.

- **Verifica arco/cammino** ($a \rightarrow b$) **con etichette**: parti dalla radice, a ogni passo calcola la posizione del figlio (sinistro/destro) con Rank_1 sul prefisso di B , leggi l'etichetta in L e confronta; segui il cammino richiesto. Costo $O(\text{lunghezza del cammino})$.
- **Stampa etichette delle foglie**: scorri B , per ogni posizione che è foglia (bit-figli a 0) emetti $L[\text{Rank}(\text{nodi fino a qui})]$. Costo $O(n)$.
- **Nodi etichettati “A” con nonno “B”**: per ogni nodo con etichetta A risali due volte al padre (via Select/formula del padre) e controlla l'etichetta del nonno; conta. Costo $O(n)$.

Il messaggio per l'esame: *non serve memorizzare puntatori*; padre/figli/foglie si ottengono con Rank e Select in $O(1)$ ciascuno, e l'intera struttura sta in $2n + o(n)$ bit.

Struttura	Spazio	Tempo per query
Rank	$m + o(m)$ bit	$O(1)$
Select	$m + o(m)$ bit	$O(1)$
FM-index Count	$nH_k + o(n \log \sigma)$ bit	$O(p)$ (se σ polylog)
FM-index Locate	$+ O(\frac{n}{\mu} \log n)$ bit	$O(\mu \cdot occ)$